

รายงานโครงงานศึกษาอิสระ

Choke Point Analysis of Attack Paths in Hybrid Infrastructure: Explainable Edge Weighting and Result Reliability

การหาจุดตัด (Choke Point) ของเส้นทางโจมตีโครงสร้างพื้นฐานแบบ Hybrid: วิธีให้น้ำหนักที่อธิบายที่มาได้ และการวัดว่าผลลัพธ์เชื่อถือได้แค่ไหน

จัดทำโดย

นายธนบดี อังกุลดี 67070065

เสนอ

ผศ.ดร. สุเมธ ประภาวัต

06016421 Information Technology Infrastructure Security

คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 1 ปีการศึกษา 2569

รายชื่อสมาชิกและหน้าที่

โครงงานนี้เป็นโครงงานเดี่ยว (สมาชิก 1 คน) จัดทำและรับผิดชอบทุกส่วนโดย นายธนบดี อังกุลดี 67070065 ตามที่เสนอไว้ในข้อเสนอโครงงาน แบ่งหน้าที่เป็น 4 ส่วนดังนี้

1. ออกแบบโครงสร้างกราฟการโจมตีและเขียนตัวสร้างกราฟจำลอง (`attackpath/graph.py`, `attackpath/gen.py`)
2. จัด lab จริงทั้งสองฝั่ง เครือข่ายภายในองค์กรและคลาวด์ (AWS ผ่าน LocalStack) แล้วดึงข้อมูลเข้า Neo4j ด้วย Cartography
3. ทำวิธีให้น้ำหนัก edge ที่อธิบายที่มาได้ และเขียนตัววิเคราะห์จุดตัด (`attackpath/weights.py`, `attackpath/chokepoints.py`)
4. ออกแบบการทดลอง วิเคราะห์ผลเชิงสถิติ และเขียนรายงานฉบับนี้

บทคัดย่อ

เครื่องมือบริหารเส้นทางโจมตี (Attack Path Management) จัดอันดับเส้นทางที่อันตรายด้วยคะแนนที่อธิบายที่มาไม่ได้ และไม่วัดว่าอันดับเปลี่ยนเท่าไรเมื่อน้ำหนักคลาดเคลื่อน โครงงานนี้สร้างกราฟการโจมตีของโครงสร้างพื้นฐานแบบ Hybrid ที่ทุกเส้นเชื่อม (edge) มีความน่าจะเป็น $p(e)$ อธิบายที่มาได้เสมอ: edge ที่มี CVE ใช้ค่า EPSS ตรง ๆ, edge ที่เป็น misconfiguration ใช้ค่าที่ fit จากความสัมพันธ์ CVSS–EPSS และ edge เชิงโครงสร้างกำหนดให้ $p = 1$ จากนั้นหาจุดตัดด้วย dominator tree เทียบกับ weighted node min-cut และวัดความเสถียรของอันดับด้วย Jaccard@k และ Kendall's τ

ทดลองบนกราฟจำลอง 216 โครงสร้าง (864 กราฟเมื่อรวมโมเดลน้ำหนัก 4 แบบ) และ lab ที่ใช้ Terraform, LocalStack, Cartography และ Neo4j ผลคือ (1) อันดับทนต่อการรบกวนน้ำหนัก (Jaccard@5 เท่ากับ 0.954 ที่ $\pm 50\%$) แต่ไม่ทนต่อ edge ที่ขาดหาย (0.49 เมื่อลบ 5% และ 0.15 เมื่อลบ 20%) (2) จุดตัดที่การันตีทุกเส้นทางพบเพียง 18 จาก 216 กราฟ (8.3%) ส่วนใหญ่ในเครือข่ายแบบ micro-segmentation (3) weighted min-cut ที่ใช้น้ำหนัก EPSS อยู่ในเซตของ dominator tree เพียง 72% ของกรณีที่เทียบได้ และฟังก์ชัน fit จาก CVSS อธิบายความแปรปรวนได้น้อย (R^2 ในสเกล logit เท่ากับ 0.219)

ข้อจำกัดคือกราฟจำลองไม่มีค่าจริงสำหรับเทียบความถูกต้อง จึงสรุปไม่ได้ว่า EPSS แม่นกว่า CVSS และ lab ใช้ Docker, nmap และ LocalStack แทน GNS3, Nessus และ AWS จริง

คำสำคัญ: จุดตัด (Choke Point), กราฟการโจมตี, Dominator Tree, EPSS, CVSS, โครงสร้างพื้นฐานแบบ Hybrid

สารบัญ

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญของปัญหา 1

1.2 ปัญหาที่ต้องการแก้ไข 2

1.3 วัตถุประสงค์ 2

1.4 ขอบเขตของโครงงาน 2

1.4.1 อยู่ในขอบเขต 2

1.4.2 นอกขอบเขต 3

1.5 ประโยชน์ที่คาดว่าจะได้รับ 3

บทที่ 2 สิ่งที่ศึกษา 4

2.1 ทฤษฎีและงานที่เกี่ยวข้อง 4

2.1.1 กราฟการโจมตี (Attack Graph) 4

2.1.2 การเสริมความมั่นคงด้วยต้นทุนต่ำสุด 5

2.1.3 Dominator Tree 5

2.1.4 CVSS และ EPSS 6

2.1.5 เครื่องมือที่เกี่ยวข้อง 7

2.1.6 ช่องว่างที่โครงงานนี้เติมเต็ม 7

2.2 การออกแบบระบบ 8

2.2.1 โมเดลกราฟการโจมตี 8

2.2.2 ที่มาของความน่าจะเป็น $p(e)$ 10

2.2.3 Rubric ของ misconfiguration 11

2.2.4 ตัวสร้างกราฟจำลอง 11

2.2.5 นิยามจุดตัด 12

2.2.6 Weighted Min-Cut 14

2.2.7 การออกแบบการทดลอง 15

2.2.8 ตัวชี้วัด 16

บทที่ 3 เนื้องาน 18

3.1 โครงสร้างโปรเจกต์ 18

3.2 ท่อประมวลผลข้อมูลและการ fit 19

3.3 ตัววิเคราะห์จุดตัด 19

3.4 การทำซ้ำผลได้ (Reproducibility) 19

3.5 Lab: Terraform, Cartography, Neo4j 20

3.6 การทดสอบซอฟต์แวร์ 21

บทที่ 4 ประเมินผล อภิปราย และวิเคราะห์ 23

4.1 ผลการประมาณค่า $p(e)$ จาก CVSS (วัตถุประสงค์ข้อ 2) 23

4.1.1 ข้อมูลและวิธีการ 23

4.1.2 ผลการ fit 24

4.1.3 ความคลาดเคลื่อนในหน่วย log 26

4.1.4 ผลต่อความสามารถในการแยกอันดับเส้นทาง 28

4.2 การเปรียบเทียบ Dominator Tree กับ Weighted Min-Cut (วัตถุประสงค์ข้อ 3) 28

4.2.1 สามนิยามของจุดตัด 29

4.2.2 จุดตัดเชิงโครงสร้างเป็นกรณีสุดขั้วของชั้นความน่าจะเป็น . . . 29

4.2.3 ความตรงกันของ dominator กับ min-cut 29

4.2.4 ระดับการแบ่งส่วนเครือข่ายกับการเกิดจุดตัด 31

4.3 ความเสถียรของอันดับเมื่อน้ำหนักถูกรบกวน (วัตถุประสงค์ข้อ 4) 32

4.4 ความเสถียรของอันดับเมื่อ edge ขาดหาย 34

4.5 การเปรียบเทียบโมเดลน้ำหนัก 4 แบบ 36

4.6 ผลของความคลาดเคลื่อนจากการ fit ต่ออันดับจุดตัด 37

4.7 สัดส่วนเส้นทางที่ข้ามโดเมน 38

4.8 ผลจาก lab จริง (วัตถุประสงค์ข้อ 5) 40

4.9 อภิปรายผลและภัยต่อความตรงของผล 42

4.10 ข้อจำกัด 44

บทที่ 5 สรุป 45

5.1 สรุปผลตามวัตถุประสงค์ 45

5.2 ข้อจำกัด 46
5.3 แนวทางพัฒนาต่อ 46

เอกสารอ้างอิง **47**

ภาคผนวก ก ตาราง Rubric ของ misconfiguration **48**

ภาคผนวก ข สคีมา edge list **50**

ภาคผนวก ค วิธีทำซ้ำผลการทดลอง **52**

ภาคผนวก ง ความแตกต่างจากข้อเสนอโครงงาน (Deviation log) **54**

สารบัญตาราง

ตารางที่ 1 การทดลองทั้งเจ็ดชุด 16

ตารางที่ 2 ผลการ fit จากเวกเตอร์ CVSS ไปเป็น EPSS (ข้อมูลทดสอบ 25%) . . . 24

ตารางที่ 3 ความคลาดเคลื่อน $|r|$ ในหน่วย nats แยกตามช่วงของ EPSS จริง . . . 27

ตารางที่ 4 ผลต่างต้นทุนขั้นต่ำ (nats) ที่ทำให้สองเส้นทางยาวเท่ากันยังรักษาอันดับไว้ได้ 28

ตารางที่ 5 สามนิยามของจุดตัดบน 864 กราฟถ่วงน้ำหนัก 29

ตารางที่ 6 สัดส่วน node ใน min-cut ที่อยู่ใน dominator set (18 โครงสร้างที่ global ไม่ว่าง) 30

ตารางที่ 7 สัดส่วนกราฟที่มี global choke point แยกตามการแบ่งส่วน (72 กราฟต่อระดับ) 31

ตารางที่ 8 Jaccard@5 เฉลี่ยเมื่อรบกวนน้ำหนัก (ทุกโมเดลน้ำหนักรวมกัน) 33

ตารางที่ 9 Jaccard@5 แยกตามโมเดลน้ำหนัก (แบบสมมาตร $\pm 50\%$ และเชิงประจักษ์ $8\times$) 33

ตารางที่ 10 ผลของการลบ edge แบบสุ่ม 34

ตารางที่ 11 Jaccard@5 ระหว่างคู่โมเดลน้ำหนัก (216 กราฟ จับคู่กัน) 36

ตารางที่ 12 ผลของการใช้ fitted p แทน EPSS จริง บน CVE edge (n=45 กราฟ, $n=200$) 38

ตารางที่ 13 สัดส่วนเส้นทางข้ามโดเมน แยกตาม target (น้ำหนัก EPSS, 216 กราฟ) . 39

ตารางที่ 14 ประเภท edge ในกราฟ lab จริง (9 node, 8 edge) 41

ตารางที่ 15 จุดตัดต่อ target ของกราฟ lab จริง 42

ตารางที่ 16 Rubric ของ misconfiguration ทั้ง 18 class 48

สารบัญรูป

รูปที่ 1 ตัวอย่างย่อส่วนของกราฟการโจมตี แสดง node แบบ (resource, privilege) และ edge สามชนิดตามแหล่งที่มาของ $p(e)$ (ตัวอย่างประกอบ ไม่ใช่กราฟจากการทดลองจริง) 9

รูปที่ 2 EPSS จริงเทียบกับค่าที่ฟังก์ชัน fit ทำนาย (log–log scale, CVE 46,360 รายการ) จุดสีแดงคือ CVE ที่มี EPSS จริงอยู่ใน 10% บนสุด 25

รูปที่ 3 การกระจายของ $|r| = |\log p_{\text{จริง}} - \log p_{\text{ทำนาย}}|$ เทียบระหว่างกลุ่ม pooled ทั้งหมดกับกลุ่ม EPSS สูงสุด 10% เส้นประคือค่ามัธยฐานของแต่ละกลุ่ม 27

รูปที่ 4 สัดส่วน node ใน weighted min-cut ที่อยู่ใน dominator set แยกตามสเกล capacity และโมเดลน้ำหนัก (18 โครงสร้างที่ global set ไม่ว่าง) . . 30

รูปที่ 5 สัดส่วนกราฟที่มี global choke point แยกตามการแบ่งส่วนเครือข่าย พร้อมช่วง ความเชื่อมั่น 95% แบบ Wilson ตัวเลขกำกับคือจำนวนกราฟที่มีจุดตัด/กราฟทั้งหมดในกลุ่ม 32

รูปที่ 6 Jaccard@5 เฉลี่ยเมื่อรบกวนน้ำหนักแบบสมมาตร (ซ้าย) และแบบเชิงประจักษ์ (ขวา) แยกตามโมเดลน้ำหนักทั้งสี่แบบ 34

รูปที่ 7 Jaccard@5 เฉลี่ย (ดิบ) เมื่อลบ edge แบบสุ่ม 5, 10 และ 20% แยกตามโมเดล น้ำหนักทั้งสี่แบบ 35

รูปที่ 8 Jaccard@5 ระหว่างคู่โมเดลน้ำหนักทั้งหกคู่ (216 กราฟ จับคู่กัน) แท่งสีแดงคือคู่ที่มี `random` (null model) อยู่ด้วย 37

รูปที่ 9 สัดส่วนเส้นทางข้ามโดเมน (น้ำหนัก EPSS) แยกตามกลุ่ม target 39

รูปที่ 10 กราฟจาก lab จริงทั้งใบ (9 node, 8 edge) ตามที่ `export_graph.py` ส่งออก จาก Neo4j/Cartography สีของ edge คือชนิด (`kind`) ไม่ใช่โดเมน 41

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

องค์กรทุกวันนี้ส่วนใหญ่ไม่ได้มีเพียงเครือข่ายภายใน (on-premise) อย่างเดียวอีกต่อไป แต่ผสมกับระบบบนคลาวด์ (cloud) จนกลายเป็นโครงสร้างพื้นฐานแบบ Hybrid ผู้โจมตีที่ยึดเครื่องแรกได้มักไม่หยุดอยู่แค่นั้น แต่เดินทางต่อไปเรื่อย ๆ จนถึงข้อมูลสำคัญ เครื่องมือกลุ่ม Attack Path Management เช่น BloodHound, Wiz และ XM Cyber ช่วยแก้ ปัญหานี้ได้ในระดับหนึ่ง ด้วยการมองทั้งระบบเป็นกราฟ แล้วหาเส้นทางที่ผู้โจมตีเดินได้ แต่เครื่องมือเหล่านี้ยังเหลือช่องว่างสำคัญสองจุด

ช่องว่างแรกคือการให้น้ำหนักแต่ละเส้นทาง เครื่องมือเชิงพาณิชย์จัดอันดับได้ว่าเส้นทางใดอันตรายที่สุด แต่คะแนนที่ใช้มักอธิบายที่มาไม่ได้ ข้อมูลช่องโหว่ในโลกจริงมักคลาดเคลื่อนไป สัก 20–30% เป็นเรื่องปกติ แต่ไม่เคยมีรายงานเลยว่าถ้าคลาดเคลื่อนขนาดนี้ อันดับที่ได้จะยัง เชื่อถือได้หรือไม่ ช่องว่างนี้ชัดเจนที่สุดบน edge ที่ข้ามโดเมน เครื่องมือฝั่ง network อย่าง RedSeal กับ Skybox วิเคราะห์เฉพาะอุปกรณ์เครือข่ายในองค์กรอย่าง router และ firewall ส่วนฝั่ง cloud อย่าง Wiz วิเคราะห์เฉพาะทรัพยากรบนผู้ให้บริการคลาวด์ ทั้งสองฝั่งเป็นคนละ เครื่องมือกัน edge ที่ข้ามระหว่างสองฝั่งจึงกลายเป็นจุดบอดร่วมของทั้งคู่ ตัวอย่างที่พบบ่อยใน โลกจริงคือไฟล์ `~/.aws/credentials` เก็บกุญแจเข้าถึงบัญชี AWS ไว้ แล้วตกค้างอยู่ บนเซิร์ฟเวอร์ในองค์กรโดยไม่ได้ตั้งใจ หรือไม่ก็การโจมตีแบบ SSRF (Server-Side Request Forgery) หลอกให้เซิร์ฟเวอร์ที่ถูกเจาะแล้วยิง request ออกไปแทนผู้โจมตี เพื่อดึง token ออกจาก IMDS (Instance Metadata Service) บริการภายในเครื่องเสมือนบนคลาวด์ที่แจก กุญแจเข้าถึงชั่วคราวให้ตัวเครื่องเองโดยไม่ต้องยืนยันตัวตนซ้ำ การโจมตีจริงมักเดินผ่านทาง เหล่านี้ ไม่ใช่ผ่านช่องโหว่ของซอฟต์แวร์โดยตรง

ช่องว่างที่สองคือการวิเคราะห์จุดตัด (choke point) ของกราฟข้ามโดเมนยังไม่เป็นระบบ: แนวคิด attack graph มีมาตั้งแต่ Sheyner ในปี 2002 และ MulVAL ในปี 2005 ส่วนการหา จุดตัดด้วย dominator tree และด้วยน้ำหนักเชิงความน่าจะเป็นต่างก็เป็นวิธีที่รู้จักกันดี แต่ยังไม่มีงานที่นำทั้งสองวิธีมาเทียบกันโดยตรงบนกราฟที่รวมทั้งสองโดเมน พร้อมวัดว่า ผลลัพธ์ที่ได้น่าเชื่อถือเพียงใด โครงงานนี้จึงเติมช่องว่างทั้งสองข้อ: ให้วิธีกำหนดน้ำหนัก ที่อธิบายที่มาได้ (รายละเอียดในบทที่ 2) และวัดความน่าเชื่อถือของอันดับจุดตัดที่ได้อย่าง เป็นระบบ (ผลการทดลองในบทที่ 4)

1.2 ปัญหาที่ต้องการแก้ไข

สรุปจากที่มาข้างต้น ปัญหาหลักที่โครงงานนี้ตั้งใจแก้มีสองข้อ ข้อแรกคือ **น้ำหนักบน edge ของ attack graph ขาดความน่าเชื่อถือ** เครื่องมือเชิงพาณิชย์ให้คะแนนความอันตราย ของแต่ละเส้นทางได้ก็จริง แต่ไม่เปิดเผยที่มาของคะแนนนั้น และไม่เคยบอกเลยว่าถ้าข้อมูลที่ใช้ คำนวณคลาดเคลื่อนไปสัก 20–30% ซึ่งเกิดขึ้นเป็นปกติกับข้อมูลช่องโหว่จริง อันดับความ อันตรายที่ได้จะยังน่าเชื่อถืออยู่หรือไม่ ข้อสองคือ **ยังไม่มีงานที่รวม เครือข่ายภายในองค์กรกับคลาวด์เข้าเป็นกราฟเดียวกันแล้ววิเคราะห์จุดตัด (choke point) อย่างเป็นระบบ** โดยใช้ทั้งวิธีเชิงโครงสร้าง (dominator tree) และเชิงความน่าจะเป็น ($-\log p$ + Dijkstra) เปรียบเทียบกันโดยตรง

1.3 วัตถุประสงค์

1. ออกแบบโมเดลกราฟการโจมตีที่ใช้ “สถานะของผู้โจมตี” เป็น node คือคู่ของทรัพยากร กับระดับสิทธิ์ (เช่น เครื่อง A ระดับ root) และครอบคลุม edge ที่ข้ามจากเครือข่ายภายใน องค์กรไปคลาวด์อย่างชัดเจน
2. ทำวิธีให้ค่า $p(e)$ ที่อธิบายที่มาได้ทุก edge ทั้ง edge ที่มี CVE (ใช้ EPSS) และ edge ที่เป็น misconfiguration (เทียบค่าจากความสัมพันธ์ CVSS–EPSS)
3. ใช้ dominator tree หาจุดตัดที่รับประกันบนกราฟที่มีทิศทาง แล้วเทียบกับผลจาก weighted min-cut ดูว่าสองวิธีให้คำตอบตรงกันหรือไม่
4. วัดว่าอันดับ choke point เปลี่ยนแปลงเท่าไรเมื่อรบกวนค่าน้ำหนักและเมื่อข้อมูลกราฟขาดหาย รวมถึงตอบว่าการใช้ EPSS ช่วยได้จริงกว่าการใช้ CVSS เฉย ๆ หรือไม่
5. สร้าง lab จริงทั้งสองฝั่ง ฝั่ง on-prem ใช้ GNS3 จำลองเครือข่ายเสมือน ร่วมกับ Nessus สแกนหาช่องโหว่ให้ได้รายการ CVE ที่พบจริงบนเครื่อง ฝั่งคลาวด์ใช้ AWS กับ Terraform สั่งสร้างทรัพยากรจากไฟล์คำสั่งโดยตรงแทนการคลิกทีละขั้นตอน แล้วดึงข้อมูลทั้งหมดเข้า Neo4j ฐานข้อมูลแบบกราฟที่เก็บเป็น node/edge ตรง ๆ ไม่ผ่านตาราง ผ่านเครื่องมือ โอเพนซอร์ส Cartography ที่สแกนทรัพยากรคลาวด์แล้วเขียนลงกราฟให้อัตโนมัติ ทั้งหมดนี้ เพื่อยืนยันว่าโมเดลใช้กับระบบจริงได้

1.4 ขอบเขตของโครงงาน

1.4.1 อยู่ในขอบเขต

- กราฟจำลองอย่างน้อย 200 กราฟสำหรับงานสถิติ บวกกราฟจริงจาก lab ทั้งฝั่ง network, cloud และแบบรวมสองฝั่ง

- การวิเคราะห์เชิงโครงสร้าง (dominator tree) และเชิงความน่าจะเป็น ($-\log p$) บนกราฟชุดเดียวกัน
- การทดสอบความทนทาน: รบกวนน้ำหนัก ± 10–50%, สุ่มลบ edge 5–20% และเปรียบเทียบโมเดลน้ำหนัก 4 แบบ
- ข้อสมมติที่ประกาศไว้ล่วงหน้า: ใช้ OR-semantics (ไม่รองรับ AND-node แบบ MulVAL), สมมติว่าผู้โจมตีไม่เสียสิทธิ์ที่ได้มาแล้ว (monotonic) และสมมติว่า edge เป็นอิสระต่อกัน (edge independence)

1.4.2 นอกขอบเขต

- การสั่งแก้ไขการตั้งค่าโดยอัตโนมัติ (auto-remediation)
- การเขียนตัวเก็บข้อมูลขึ้นใหม่แทน Cartography
- การตรวจจับแบบ real-time
- การรองรับ AND-node เต็มรูปแบบ

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. วิธีให้น้ำหนัก edge ที่อธิบายที่มาได้ทุก edge พร้อมเกณฑ์เทียบค่าที่คนอื่นทำซ้ำได้
2. ตัวเลขว่าน้ำหนักคลาดเคลื่อนได้เปอร์เซ็นต์เท่าไรก่อนอันดับ choke point จะเชื่อถือไม่ได้ (รายงานด้วย Kendall's τ และสัดส่วนการซ้อนทับของอันดับต้น ๆ)
3. อัตราที่ dominator tree กับ weighted min-cut ให้คำตอบไม่ตรงกัน ข้อมูลแบบนี้ เครื่องมือเชิงพาณิชย์ในปัจจุบันไม่เคยรายงานเลย
4. คำตอบว่าต้องแบ่ง segment เครือข่ายละเอียดแค่ไหนจึงจะเกิดจุดตัดที่รับประกันได้จริง และสัดส่วนของเส้นทางอันตรายที่วิ่งผ่าน edge ข้ามโดเมน
5. ตัววิเคราะห์และตัวสร้างกราฟจำลองแบบ open source พร้อมชุดข้อมูลผลการทดลองและรายงานฉบับสมบูรณ์

บทที่ 2 สิ่งที่ศึกษา

บทนี้แบ่งเป็นสองส่วน ส่วนแรก (หัวข้อ 2.1) ทบทวนแนวคิดและงานวิจัยที่โครงงานนี้ ต่อยอดมา เพื่อให้เห็นว่าแต่ละชิ้นส่วนของระบบมีที่มาจากไหนและทำไมจึงเลือกแบบนี้ ส่วนที่สอง (หัวข้อ 2.2) อธิบายว่าโครงงานนี้ออกแบบระบบขึ้นมาอย่างไรจริง ๆ ตั้งแต่โมเดลกราฟไปจนถึงตัวชี้วัดที่ใช้ตัดสินผล

2.1 ทฤษฎีและงานที่เกี่ยวข้อง

2.1.1 กราฟการโจมตี (Attack Graph)

ลองนึกภาพสถานการณ์จริงสักอย่างหนึ่ง: ผู้โจมตีเจาะเข้าเครื่องหนึ่งได้สิทธิ์ผู้ใช้ทั่วไปก่อน จากนั้นใช้ช่องโหว่อีกตัวยกระดับขึ้นเป็นสิทธิ์ผู้ดูแลระบบ แล้วอาศัยสิทธิ์นั้นไปขโมยกุญแจเข้าถึง ระบบคลาวด์ที่หลงเหลืออยู่บนเครื่องนั้น สุดท้ายจึงเข้าถึงข้อมูลสำคัญบนคลาวด์ได้สำเร็จ แต่ละ ขั้นตอนข้างต้นคือการ “ขยับจากสถานะหนึ่งไปอีกสถานะหนึ่ง” ของผู้โจมตี แนวคิดพื้นฐานของ โครงงานนี้คือการนำลำดับขั้นตอนแบบนี้ของทั้งระบบมาวาดเป็นกราฟหนึ่งใบ โดยแทน “สถานะที่ ผู้โจมตียืนอยู่ได้” แต่ละแบบด้วย node หนึ่งจุด (เช่น “มีสิทธิ์ user บนเครื่อง A”) และแทน “การกระทำหนึ่งครั้งที่พาไปสู่สถานะถัดไป” ด้วย edge หนึ่งเส้น (เช่น การ exploit ช่องโหว่ หนึ่งตัว) เมื่อวาดกราฟครบทั้งระบบแล้ว คำถามที่ตอบได้ทันทีคือ “มีเส้นทางไหนบ้างที่พาผู้โจมตี จากจุดเริ่มต้นไปถึงเป้าหมายได้” กราฟแบบนี้เรียกว่า attack graph

แนวคิดนี้มีรากฐานจากงานของ Sheyner และคณะ [1] พวกเขาเป็นกลุ่มแรก ๆ ที่ทำให้คอมพิวเตอร์สร้างกราฟแบบนี้ขึ้นมาเองโดยอัตโนมัติ ป้อนข้อมูลช่องโหว่ของระบบเข้าไป แล้วให้โปรแกรมไล่หาเส้นทางโจมตีที่เป็นไปได้ทั้งหมดออกมาเอง งานชิ้นนี้ยืมเทคนิคที่แต่เดิมใช้ ตรวจสอบว่าโปรแกรมคอมพิวเตอร์เขียนถูกต้องหรือไม่ เรียกว่า model checking มาใช้กับงาน ความมั่นคงปลอดภัยแทน ข้อจำกัดสำคัญคือยิ่งเครือข่ายใหญ่ จำนวนเส้นทางที่เป็นไปได้ก็ยิ่งเพิ่มขึ้น เร็วมาก วิธีนี้จึงใช้ได้จริงเฉพาะเครือข่ายขนาดเล็กเท่านั้น

ปัญหาเรื่องต้นทุนการคำนวณนี้เองที่ Ou และคณะ [2] แก้ไขด้วยเครื่องมือชื่อ MulVAL โดยเปลี่ยนไปเขียนกฎการโจมตีด้วย Datalog (ภาษาสำหรับเขียนกฎเชิงตรรกะแบบ “ถ้า...แล้ว...”) แทนการไล่ตรวจทีละสถานะแบบเดิม ทำให้คำนวณได้เร็วขึ้นมาก งานนี้ยังเพิ่ม แนวคิดสำคัญอีกอย่างเข้าไปในกราฟ เรียกว่า **AND-node** สำหรับกรณีที่ต้องมีหลาย เงื่อนไขเกิดขึ้น*พร้อมกัน*การโจมตีจึงจะสำเร็จ เช่น ต้องมีทั้งช่องโหว่ซอฟต์แวร์ *และ* การตั้งค่าไฟร์วอลล์ที่ผิดพลาดพร้อมกันทั้งคู่ ขาด

อย่างใดอย่างหนึ่งไปก็ไปต่อไม่ได้ ต่างจาก node ปกติที่โครงงานนี้ใช้ เรียกว่า OR-node มีทางใดทางหนึ่งสำเร็จก็ไปต่อได้แล้ว ไม่ต้องครบทุกทาง

โครงงานนี้เลือกใช้ OR-node ล้วน ไม่รองรับ AND-node เลย ตามที่ประกาศไว้ในขอบเขตของ โครงงาน (หัวข้อ 1.4) เพราะทำให้คำนวณง่ายและคาดเดาเวลาที่ใช้ได้แน่นอน กว่า (เหตุผลเชิงเทคนิคอยู่ในหัวข้อ 2.1.3) ข้อแลกเปลี่ยนคือโครงงานนี้ แทนเงื่อนไข “ต้องมีหลายอย่างพร้อมกัน” แบบ AND-node ไม่ได้เลย ข้อจำกัดข้อนี้รู้ตัวมาตั้งแต่ ต้น และย้ำอีกครั้งในหัวข้อ 4.10

2.1.2 การเสริมความมั่นคงด้วยต้นทุนต่ำสุด

เมื่อมีกราฟการโจมตีแล้ว คำถามที่ตามมาตามธรรมชาติคือ “แล้วจะป้องกันอย่างไร” สายงาน วิจัยอีกสายหนึ่งตอบคำถามนี้ตรง ๆ ว่า “ต้องปิดกั้นให้น้อยที่สุดกี่จุด เครือข่ายจึงจะปลอดภัยขึ้น” Noel และคณะ [3] ตอบคำถามนี้บนกราฟการโจมตี โดยมองปัญหาคล้ายกับการตัด ถนนหรือตัดสายไฟ: หา node หรือ edge ชุดเล็กที่สุดเท่าที่จะเป็นไปได้ ที่ถ้าตัดออกแล้วผู้โจมตี จะไปไม่ถึงเป้าหมายอีกต่อไป ปัญหาแบบนี้เรียกว่า *min-cut* คือการตัดที่ต้นทุนต่ำที่สุด โครงงานนี้นำแนวคิดนี้มาใช้โดยตรง แล้วเทียบผลกับ dominator tree (หัวข้อถัดไป) เพื่อตอบ วัตถุประสงค์ข้อ 3 รายละเอียดอยู่ในหัวข้อ 2.2.6 และ 4.2 สิ่งที่โครงงานนี้ต่างจากงานของ Noel คือ Noel ตัด *edge* ออก ตัดเส้นทางส่วน โครงงานนี้ตัด *node* ออกแทน ตัดจุดยืนของผู้โจมตีเอง โดยใช้เทคนิคที่เรียกว่า node-splitting และเปรียบเทียบผลที่ได้กับจุดตัดแบบไม่ใช้น้ำหนักเลย คือจุดตัดเชิงโครงสร้าง ล้วน ๆ ไปพร้อมกันด้วย

2.1.3 Dominator Tree

ลองนึกถึงเกาะที่เชื่อมกับแผ่นดินใหญ่ด้วยสะพานเพียงสะพานเดียว ไม่ว่าจะเดินทางด้วยเส้นทางไหนบนเกาะนั้น ถ้าจะออกไปแผ่นดินใหญ่ก็ต้องผ่านสะพานเส้นนี้เสมอ สะพานแบบนี้คือตัวอย่าง ที่เข้าใจง่ายที่สุดของสิ่งที่เรียกว่า *dominator* วิธีที่สองที่โครงงานนี้ใช้หาจุดตัดคือ dominator tree ตอบคำถามว่า “node ใดบ้างที่ทุกเส้นทางจาก root ไปยัง node เป้าหมายหนึ่ง ๆ ต้องผ่านเสมอ” node แบบนั้นเรียกว่า dominator ของ node เป้าหมายนั้น

การหา dominator ทั้งหมดในกราฟพร้อมกันเป็นปัญหาคลาสสิกของทฤษฎีกราฟที่มีทิศทาง มี อัลกอริทึมมาตรฐานที่คำนวณได้เร็วมากอยู่แล้ว โครงงานนี้เรียกใช้ผ่านฟังก์ชันสำเร็จรูป `networkx.immediate_dominators` ซึ่งมาจากงานของ Lengauer และ Tarjan [4] เดิมทีอัลกอริทึมนี้ถูกออกแบบมาสำหรับวิเคราะห์โปรแกรม คอมพิวเตอร์ ไม่ใช่งานความมั่นคงปลอดภัย โดยใช้หาว่าในขั้นตอนการทำงานของโปรแกรม ซึ่ง เขียนแทนด้วยกราฟของลำดับการทำงานของโค้ดที่เรียกว่า control-flow graph คำสั่งใดบ้าง ที่การไหลของโปรแกรมต้องผ่านเสมอ

ไม่ว่าจะวิ่งไปทางไหน โครงงานนี้นำอัลกอริทึมเดียวกันมาใช้ กับ attack graph แทน เพียงแค่เปลี่ยนความหมายจาก “จุดที่โปรแกรมต้องผ่าน” เป็น “จุดที่ ผู้โจมตีต้องผ่าน” เท่านั้น ตัวอัลกอริทึมเองไม่ได้เปลี่ยนแปลงอะไรเลย

2.1.4 CVSS และ EPSS

ถัดมาคือสองมาตรฐานการให้คะแนนช่องโหว่ที่โครงงานนี้ใช้สร้างความน่าจะเป็นบน edge มาตรฐานแรกคือ Common Vulnerability Scoring System (CVSS) [6] ให้คะแนน ความรุนแรงของช่องโหว่หนึ่งรายการจากคุณสมบัติของตัวช่องโหว่เองล้วน ๆ เช่น ต้องโจมตีจาก เครือข่ายหรือต้องอยู่เครื่องเดียวกัน ต้องมีสิทธิ์อยู่ก่อนหรือไม่ แต่ CVSS ไม่ได้บอกเลยว่า ช่องโหว่นั้นมีโอกาสถูกใช้โจมตีจริงในโลกมากน้อยเพียงใด

CVSS เขียนคะแนนออกมาเป็นเวกเตอร์ตัวย่อ 8 ตัว โครงงานนี้อ้างถึงเวกเตอร์แบบนี้ซ้ำตลอดทั้งรายงาน (เช่นในรูปแบบ `AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`) แบ่งได้เป็น สองกลุ่มตามคำถามที่แต่ละกลุ่มตอบ กลุ่มแรกตอบว่า “ผู้โจมตีต้องมีเงื่อนไขอะไรบ้างก่อนจะโจมตีสำเร็จ”:

- **AV** (Attack Vector) – ต้องโจมตีจากที่ไหน: ผ่านเครือข่าย, ต้องอยู่เครื่อง เดียวกัน, หรือต้องอยู่ใกล้ชิดทางกายภาพ
- **AC** (Attack Complexity) – โจมตีได้ง่าย ๆ หรือต้องมีเงื่อนไขพิเศษประกอบ
- **PR** (Privileges Required) – ต้องมีสิทธิ์อะไรติดตัวอยู่ก่อนแล้วบ้าง
- **UI** (User Interaction) – ต้องหลอกให้เหยื่อคลิกหรือทำอะไรสักอย่างด้วยหรือไม่

กลุ่มที่สองตอบคำถามต่างออกไป คือ “ถ้าโจมตีสำเร็จแล้ว ความเสียหายกระทบไปถึงไหน”:

- **S** (Scope) – ผลกระทบลามออกไปถึงทรัพยากรอื่นนอกขอบเขตเดิมหรือไม่ (**C** คือ Changed แปลว่าลามออกไป, **U** คือ Unchanged แปลว่าไม่ลาม)
- **C/I/A** (Confidentiality/Integrity/Availability) – ความลับ, ความถูกต้อง และความพร้อมใช้งานของข้อมูลถูกกระทบมากน้อยแค่ไหน แต่ละด้านมีค่าเป็น **N** (ไม่กระทบ), **L** (กระทบน้อย) หรือ **H** (กระทบมาก)

เวกเตอร์ชุดนี้คือสิ่งที่ rubric ในหัวข้อ 2.2.3 ต้องกำหนดให้ misconfiguration แต่ละคลาสและเป็นตัวแปรต้น 8 ตัวที่ใช้ fit หาค่า EPSS ในหัวข้อ 3.2

ช่องว่างตรงนี้คือสิ่งที่มาตรฐานที่สอง Exploit Prediction Scoring System (EPSS) [5, 7] เข้ามาเติมเต็ม โดยฝึกโมเดลทำนายจากข้อมูลการโจมตีจริง ที่สังเกตได้ (telemetry) แล้วให้คะแนน

ความน่าจะเป็นว่า CVE หนึ่งรายการจะถูกใช้โจมตีจริง ภายในกรอบเวลาหนึ่ง พูดสั้น ๆ คือสองคะแนนนี้วัดคนละเรื่องกัน: CVSS ตอบว่า “ถ้าถูกใช้แล้ว รุนแรงแค่ไหน” ส่วน EPSS ตอบว่า “มีโอกาสถูกใช้ที่ไหนสักแห่งในโลกแค่ไหน” ข้อควรระวังคือ EPSS ไม่ได้ตอบว่า “มีโอกาสสำเร็จแค่ไหนเมื่อผู้โจมตียืนอยู่ต้นทางแล้ว” นั่นเป็นคนละคำถามกัน โดยสิ้นเชิง และเป็นเหตุผลที่โครงงานนี้ต้องสร้างฟังก์ชันแปลงค่าระหว่างสองคะแนนขึ้นมาเอง แทนที่จะหยิบตัวใดตัวหนึ่งมาใช้แทนกันตรง ๆ (รายละเอียดวิธีแปลงอยู่ในหัวข้อ 2.2.2 ผลที่ได้อยู่ในหัวข้อ 4.1) เหตุผลที่ต้องแปลงคือ edge ที่เป็น misconfiguration ไม่มี CVE ผูกอยู่เลย จึงไม่มีคะแนน EPSS ให้หยิบมาใช้ได้ตรง ๆ ตั้งแต่ต้น

2.1.5 เครื่องมือที่เกี่ยวข้อง

ในเชิงอุตสาหกรรม มีเครื่องมือกลุ่ม Attack Path Management ที่ใช้แนวคิด attack graph ในระดับใช้งานจริงอยู่แล้ว เช่น BloodHound (เน้นวิเคราะห์ Active Directory), Wiz (เน้น คลาวด์) และ XM Cyber แต่เครื่องมือเหล่านี้มักมุ่งไปที่ฝั่งใดฝั่งหนึ่งเป็นหลัก เลือกวิเคราะห์ เฉพาะ on-prem หรือเฉพาะ cloud ไม่ใช่ทั้งสองฝั่งรวมกัน ที่สำคัญกว่านั้นคือไม่มีตัวไหนเปิดเผย เลยว่าคะแนนความอันตรายที่ใช้จัดอันดับมาจากการคำนวณอะไร นี่คือช่องว่างที่โครงงานนี้ตั้งใจ เติมเต็ม อธิบายต่อในหัวข้อถัดไป

ส่วนเครื่องมือภายนอกตัวเดียวที่โครงงานนี้นำมาใช้จริงคือ Cartography เครื่องมือ โอเพนซอร์สที่ดึงข้อมูลโครงสร้างพื้นฐานคลาวด์เข้า Neo4j ให้อัตโนมัติ นำมาใช้เป็นตัวเก็บข้อมูล ฝั่งคลาวด์ของ lab จริง (หัวข้อ 3.5) โดยไม่เขียนตัวเก็บข้อมูลขึ้นใหม่เองเลย ตามที่ประกาศไว้ในขอบเขตของโครงงาน

2.1.6 ช่องว่างที่โครงงานนี้เติมเต็ม

สรุปจากงานวิจัยและเครื่องมือทั้งหมดข้างต้น โครงงานนี้ไม่ได้เสนอวิธีสร้างหรือวิเคราะห์ attack graph แบบใหม่ แต่เติมช่องว่างสามจุดที่ไม่มีงานไหนตอบพร้อมกันทั้งหมด จุดแรกคือ วิธีให้น้ำหนัก edge ที่อธิบายที่มาได้ทุกเส้น แยกวิธีคำนวณตามชนิดของ edge: ถ้ามี CVE จริง ใช้ EPSS [5, 7] ตรง ๆ ถ้าเป็น misconfiguration ที่ไม่มี CVE ใช้ค่าที่ fit มาจากเวกเตอร์ CVSS [6] แล้วอ้างอิง CWE และ CIS Benchmark กำกับไว้เป็น หลักฐานประกอบทุกคลาส CWE ย่อมาจาก Common Weakness Enumeration [8] คือรายการมาตรฐานที่ MITRE จัดทำไว้สำหรับจำแนกประเภทของจุดอ่อนในซอฟต์แวร์หรือการ ตั้งค่า แต่ละรายการมีหมายเลขกำกับ เช่น CWE-521 คือ “รหัสผ่านที่ไม่รัดกุมพอ” ส่วน CIS Benchmark คือคู่มือมาตรฐานการตั้งค่าระบบให้ปลอดภัยที่จัดทำโดย Center for Internet Security [9] ใช้อ้างอิงว่าการตั้งค่าแบบใดถือว่าหละหลวมตามมาตรฐานอุตสาหกรรม ไม่ใช่ดุลพินิจของผู้ทำโครงงานเอง

จุดที่สองคือการเทียบ dominator tree [4] กับ weighted min-cut [3] บนกราฟชุดเดียวกันโดยตรง แทนที่จะเลือกใช้วิธีใดวิธีหนึ่งเพียง อย่างเดียวตามที่งานก่อนหน้าทำ จุดที่สามคือการวัดความน่าเชื่อถือของอันดับจุดตัดอย่างเป็น ระบบ ทั้งเมื่อรบกวนน้ำหนักและเมื่อข้อมูลขาดหาย วัดด้วย Jaccard@k และ Kendall's τ เป็นคำถามที่เครื่องมือเชิงพาณิชย์ในหัวข้อก่อนไม่เคยตอบมาก่อนเลย

2.2 การออกแบบระบบ

จากทฤษฎีในหัวข้อก่อน หัวข้อนี้อธิบายว่าโครงงานนี้นำแนวคิดเหล่านั้นมาสร้างเป็นระบบจริง อย่างไร เริ่มจากโมเดลกราฟ (หัวข้อ 2.2.1) ไปจนถึงตัวชี้วัดที่ใช้อ่านผลการ ทดลอง (หัวข้อ 2.2.8)

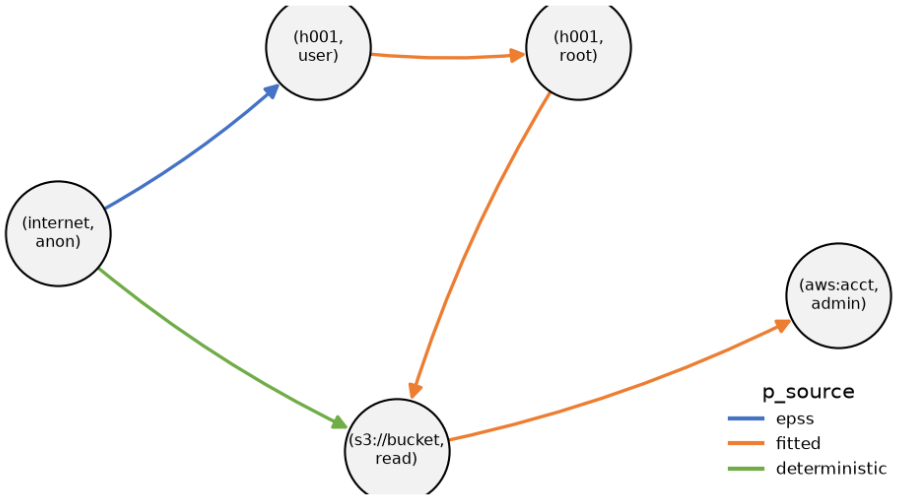
2.2.1 โมเดลกราฟการโจมตี

node ของกราฟในโครงงานนี้คือคู่ `(resource, privilege)` เช่น `(h001, root)` หมายถึง "ผู้โจมตีถือสิทธิ์ root บนเครื่อง h001" หรือ `(s3://bucket, read)` หมายถึง "ผู้โจมตีอ่านข้อมูลจาก bucket นั้นได้" การเข้ารหัสทั้งทรัพยากรและสิทธิ์ไว้ใน node เดียวกันแบบนี้ แทนที่จะแยกเป็นสองมิติ ทำให้สถานะของผู้โจมตีสมบูรณ์ในตัวมันเอง เวลาจะ ตรวจว่า edge หนึ่งเส้นใช้ได้หรือไม่ ดูที่ node ต้นทางเพียงอย่างเดียวก็พอ ไม่ต้องย้อนไปดูว่า ผู้โจมตีถือสิทธิ์อะไรมาก่อนหน้านั้นจากที่ไหน

edge หนึ่งเส้นแทนการกระทำหนึ่งครั้งของผู้โจมตี เช่น หนึ่ง exploit หรือหนึ่งการเดินเครือข่าย ไปอีกจุดหนึ่ง edge แบ่งเป็นสามชนิดตามแหล่งที่มาของความน่าจะเป็น อธิบายละเอียดในหัวข้อ ถัดไป (หัวข้อ 2.2.2) และทุกเส้นมีคุณสมบัติ `domain` กำกับไว้ว่าอยู่ ฝั่ง `onprem`, `cloud` หรือ `cross` คือข้ามจากฝั่งหนึ่งไปอีกฝั่ง edge ที่มี `domain="cross"` คือใจกลางของวัตถุประสงค์ข้อ 1 เพราะโครงงานนี้ตั้งใจศึกษา เส้นทางที่ข้ามจากเครือข่ายภายในองค์กรไปคลาวด์โดยเฉพาะ ตัวตรวจสอบโครงสร้าง (`validate_structure`) จึงบังคับว่ากราฟแบบ hybrid ทุกใบต้องมี edge ข้ามโดเมน อย่างน้อยหนึ่งเส้นเสมอ ไม่เช่นนั้นจะไม่ถือว่าเป็นกราฟ hybrid จริง

รูปที่ 1 แสดงตัวอย่างย่อส่วนที่ประกอบขึ้นจากแนวคิดทั้งหมดข้างต้น ผู้โจมตีเริ่มจาก `(internet, anon)` แล้วไล่เจาะ CVE จริงบนเครื่อง `h001` จน ได้สิทธิ์ `user` (edge สีน้ำเงิน, `p_source=epss`) จากนั้นใช้รหัสผ่าน admin ที่ตั้งซ้ำกันทุกเครื่องยกระดับเป็น `root` (edge สีส้ม, `p_source=fitted`) ขณะเดียวกันก็เข้าถึง S3 bucket ที่เปิดอ่านสาธารณะได้โดยตรงจากอินเทอร์เน็ตเลย (edge สีเขียว, `p_source=deterministic`, ไม่ต้องเจาะอะไรก่อน) แล้วอ่านกุญแจ AWS ที่ หลงเหลืออยู่บนเครื่อง `h001` เพื่อไปสวมสิทธิ์ admin

บนบัญชีคลาวด์ในที่สุด สีของ edge ที่ต่างกันในรูปนี้ไม่ได้แปลว่าอันตรายมากน้อยต่างกันแต่อย่างใด แต่บอกว่าความน่าจะ เป็น $p(e)$ ของแต่ละเส้นมาจากคนละแหล่งข้อมูล การแยกแหล่งที่มาให้ชัดเจนแบบนี้คือสิ่งที่ วัตถุประสงค์ข้อ 2 กำหนดไว้โดยตรง



รูปที่ 1 ตัวอย่างย่อส่วนของกราฟการโจมตี แสดง node แบบ (resource, privilege) และ edge สามชนิดตามแหล่งที่มาของ $p(e)$ (ตัวอย่างประกอบ ไม่ใช่กราฟจากการทดลองจริง)

โมเดลทั้งหมดนี้อยู่ภายใต้สามสมมติฐานที่ประกาศไว้ตั้งแต่ขอบเขตของโครงงาน (หัวข้อ 1.4) ได้แก่ OR-semantics (node จะไปถึงได้เมื่อมี predecessor อย่างน้อย หนึ่งตัวไปถึง ไม่ต้องครบทุกตัวแบบ AND-node ของ MulVAL), monotonic (ผู้โจมตีไม่มีวันเสีย สิทธิ์ที่ได้มาแล้ว) และ edge independence (ความน่าจะเป็น $p(e)$ ของแต่ละ edge เป็นอิสระ ต่อกัน ไม่ขึ้นกับ edge อื่น) ภายใต้สมมติฐานข้อสุดท้ายนี้ ความน่าจะเป็นที่เส้นทางหนึ่งจะสำเร็จ ทั้งเส้นคือผลคูณของ $p(e)$ ของทุก edge บนเส้นทางนั้น การแปลงแต่ละ $p(e)$ เป็น $w(e) = -\log p(e)$ ก่อนจึงเปลี่ยนการคูณให้กลายเป็นการบวก (ต้นทุนของเส้นทางคือผลรวม ของ $w(e)$) อัลกอริทึม Dijkstra มาตรฐานใช้ได้โดยตรง และยังรักษาความหมายไว้ถูกต้องด้วย: edge ที่ p ใกล้ 1 เกือบสำเร็จแน่ จะได้ w ใกล้ 0 คือต้นทุนถูก ส่วน edge ที่ p ใกล้ 0 แทบไม่สำเร็จ จะได้ w สูงมาก คือต้นทุนแพง “เส้นทางที่ถูกที่สุด” ตามน้ำหนักนี้จึงตรงกับ “เส้นทางที่มีโอกาสสำเร็จสูงที่สุด” พอดี

สมมติฐานเหล่านี้มีผลตามมาที่สำคัญหนึ่งข้อ ต้องเข้าใจให้ถูกตั้งแต่ต้น: ผลลัพธ์ของ shortest path ด้วย Dijkstra (อัลกอริทึมมาตรฐานที่หาเส้นทางต้นทุนรวมต่ำสุดจากจุดหนึ่งไปอีกจุดหนึ่ง บนกราฟที่มีน้ำหนักบวกบน edge ทุกเส้น ไล่ขยายจาก node ที่ต้นทุนสะสมน้อยที่สุดก่อนเสมอ) บนน้ำหนัก $w(e) = -\log p(e)$ บอกได้แค่ว่า “ถ้าผู้โจมตีเดินตามเส้นทางที่ง่ายที่สุดเพียงเส้นเดียว ต้นทุนจะเท่าไร” เท่านั้น ไม่ใช่โอกาสที่ระบบทั้งระบบจะถูกเจาะจริง ($P(\text{compromise})$)

ในความเป็นจริงภายใต้ OR-semantics ยังมีเส้นทางอื่นที่ไปถึง เป้าหมายเดียวกันได้อีกหลายเส้น แต่ละเส้นก็เป็นโอกาสเพิ่มเติมให้ผู้โจมตีสำเร็จ โอกาสที่แท้จริง จึงต้องสูงกว่าที่คำนวณจากเส้นทางเดียวเสมอ ต้องรวมทุกเส้นทางเข้าด้วยกัน ไม่ใช่ดูเฉพาะเส้น ที่ถูกที่สุดเส้นเดียว ตัวเลขที่ได้จาก Dijkstra จึงเรียกว่า **ขอบเขตล่างของต้นทุน ผู้โจมตี** (lower bound) พูดง่าย ๆ ก็คือต้นทุนจริงของผู้โจมตีจะไม่มีทางแพงไปกว่าตัวเลขนี้เลย มีแต่จะเท่ากันหรือถูกกว่า เพราะเส้นทางสำรองอื่น ๆ ที่ยังไม่ได้นับ มีแต่จะทำให้ผู้โจมตีทำงาน ง่ายขึ้น ไม่ใช่ยากขึ้น ตัวเลขนี้จึงเป็นค่าประเมินความเสี่ยงต่ำสุดที่เป็นไปได้ ไม่ใช่ค่าความเสี่ยง จริงทั้งหมด

2.2.2 ที่มาของความน่าจะเป็น $p(e)$

เมื่อ edge แต่ละเส้นต้องมีความน่าจะเป็นกำกับ คำถามต่อมาคือค่านั้นมาจากไหน และ ทำอย่างไรจึงจะตรวจสอบย้อนกลับได้ว่าทำไมจึงเป็นค่านี้ นี่คือข้อบังคับของวัตถุประสงค์ข้อ 2 วิธีแก้ของโครงงานนี้คือให้ทุก edge มีคุณสมบัติ `p_source` หนึ่งในสี่ค่า พร้อม `p_input` ที่ระบุแหล่งที่มาตรงตัวของค่า p นั้น:

- `epss` – ใช้กับ edge ที่มี CVE จริงผูกอยู่ นำคะแนน EPSS ของ CVE นั้นมาใช้ ตรง ๆ
- `fitted` – ใช้กับ edge ที่เป็น misconfiguration (ไม่มี CVE ให้ใช้) คำนวณจาก ฟังก์ชันที่ fit ไว้ระหว่างเวกเตอร์ CVSS กับ EPSS (รายละเอียดใน หัวข้อ 2.2.3 และผลการ fit ใน หัวข้อ 4.1)
- `deterministic` – ใช้กับ edge เชิงโครงสร้างหรือ reachability ล้วน คือ edge ที่มีอยู่แล้วโดยการตั้งค่าของระบบเอง ไม่ต้องอาศัยการเจาะช่องโหวใด ๆ เพิ่มเติม ตัวอย่างเช่น security group (กฎไฟร์วอลล์ของ AWS) ที่เปิดรับทุก IP (`0.0.0.0/0`), เครือข่ายแบบ flat VLAN (ทุกเครื่องอยู่วงเดียวกัน คุยกันได้โดยตรงโดยไม่มีไฟร์วอลล์กั้น), VPC peering (การเชื่อมสองเครือข่ายคลาวด์ส่วนตัวเข้าด้วยกันให้คุยกันได้โดยตรง), forest trust (การ ตั้งค่าไว้วางใจกันระหว่างสอง domain ของ Active Directory ทำให้ user ฝั่งหนึ่งถูกมองเห็น และอาจใช้สิทธิ์ข้ามไปอีกฝั่งได้) และ role assume (การที่ AWS IAM role หนึ่งได้รับอนุญาตล่วงหน้าให้สวมสิทธิ์ของอีก role หนึ่งได้ทันทีโดยไม่ต้องยืนยันตัวตนใหม่) ทุกตัวอย่างข้างต้นให้ $p = 1.0$ เสมอ เพราะ edge พวกนี้ตัดสินแค่ว่า *edge มีอยู่หรือไม่* ไม่ได้ตัดสินว่าเจาะสำเร็จหรือไม่ ถ้าใส่ $p < 1$ ให้ edge ชนิดนี้ จะกลายเป็นการประเมินความ เสี่ยงต่ำกว่าความเป็นจริง
- `expert` – ค่าสำรองที่ประกาศไว้ชัดเจนว่าใช้เมื่อใด ได้แก่กรณี CVE ไม่มีคะแนน EPSS ในฐานข้อมูล หรือ misconfiguration class นั้นยังไม่ได้ fit (เช่นในโมเดลเปรียบเทียบ `cvss`/`uniform`/`random` ที่ใช้ในหัวข้อ 2.2.7) ไม่ใช่ค่าเริ่มต้นที่แอบใส่โดยไม่บอก

การแบ่งเป็นสามชนิดแบบนี้ ไม่ยุบเหลือสองหรือรวมเป็นหนึ่ง เป็นการตัดสินใจที่ตั้งใจ เพราะ edge เชิงโครงสร้างกับ edge เชิง exploit มีความหมายต่างกันโดยสิ้นเชิง ถ้าผสมสองความหมาย นี้เข้าด้วยกัน การตีความ $w(e) = -\log p(e)$ จะผิดเพี้ยนไปทันที (รายละเอียดอยู่ในภัยต่อความ ตรงของผลข้อ 5 ในหัวข้อ 4.9)

2.2.3 Rubric ของ misconfiguration

หัวข้อก่อนบอกว่า edge ชนิด `fitted` ต้องแปลงจากเวกเตอร์ CVSS เป็น $p(e)$ แต่ก่อน จะแปลงได้ ต้องมีคนกำหนดก่อนว่าแต่ละ misconfiguration ควรมีเวกเตอร์ CVSS หน้าตาแบบ ไหน นี่คือหน้าที่ของ rubric (ตารางเต็มอยู่ในภาคผนวก 5.3) กำหนดเวกเตอร์ให้ misconfiguration ทั้งหมด 18 class โดยยึดกฎเดียวตลอดทั้งตาราง เรียกว่า “ให้คะแนนเทียบ node ต้นทาง” กล่าวคือ ปกติ CVSS ให้คะแนนจากมุมมองผู้โจมตีที่อยู่บนอินเทอร์เน็ตและยังไม่มี อะไรในมือเลย แต่ rubric นี้ให้แต่ละเวกเตอร์อธิบาย edge เดียว $u \to v$ โดยสมมติว่าผู้โจมตี *ยืนอยู่ที่สถานะ u แล้ว* ตัวอย่างเช่น `aws-key-on-host` ได้ค่า `AV:L/PR:L` ไม่ใช่ `AV:N/PR:N` เพราะกฎนี้บังคับว่าผู้โจมตีต้องมี shell บน เครื่องนั้นอยู่ก่อนแล้ว จึงจะอ่านไฟล์ credential ได้

ทั้ง 18 class แบ่งออกเป็นสองกลุ่มด้วยกฎง่าย ๆ ข้อเดียว: ดูว่า impact ทั้งสามด้าน (C/I/A) เป็น `N` ครบทุกตัวหรือไม่ ถ้าใช่ แปลว่า edge นั้นให้แค่ *reachability* (แค่เดินไป ถึง ไม่ได้ทำอะไรต่อ) ไม่ใช่ exploit จริง จึงจัดเป็น `deterministic` (มี 4 class แบบนี้) และไม่ป้อนเข้าฟังก์ชัน fit เลย เพราะ CVE ที่เผยแพร่จริงทุกตัวมี impact อย่างน้อยหนึ่งด้านที่ ไม่ใช่ `N` เวกเตอร์แบบ all-N จึงอยู่นอกขอบเขตข้อมูลที่ฟังก์ชัน fit เคยเห็นมาก่อน ถ้าไม่ใช่ all-N แปลว่าเป็น exploit หรือ abuse ก้าวหนึ่งจริง จัดเป็น `fitted` (มี 14 class แบบนี้) แต่ละแถวของทั้ง 18 class อ้างอิง CWE และ/หรือ CIS Benchmark ที่ตรงกับคลาส นั้นกำกับไว้ด้วย เพื่อให้ใครก็ตามตรวจสอบย้อนกลับได้ว่าทำไมจึงกำหนดเวกเตอร์แบบนี้ ไม่ใช่ ดุลพินิจที่ไม่มีการบันทึกไว้

2.2.4 ตัวสร้างกราฟจำลอง

เมื่อมีทั้งโมเดลกราฟและวิธีให้น้ำหนักแล้ว ส่วนถัดไปคือเครื่องมือที่สร้างกราฟตัวอย่างขึ้นมาจำนวนมากสำหรับทดลอง ตัวสร้างกราฟ (`attackpath/gen.py`) รับพารามิเตอร์สี่ตัว ได้แก่ ขนาดกราฟ $n \in \{50, 200, 800\}$ node, ความหนาแน่นของ edge ข้ามโดเมน (`low`/`med`/`high`), ระดับการแบ่งส่วนเครือข่าย (`flat`/`3tier`/`micro`) และ seed สำหรับสุ่ม

เมื่อสร้างกราฟ node ประมาณ 60% จะอยู่ฝั่ง on-prem และ 40% อยู่ฝั่ง cloud ฝั่ง on-prem แต่ละเครื่องมีสอง node เสมอคือ `user` และ `root` เชื่อมกันด้วย edge ชนิด `cve`

จริง สุ่มเลือกจาก pool ของ CVE ที่มีทั้ง vector และ EPSS พร้อมกันในข้อมูลที่ดึง มา อธิบายไว้ใน หัวข้อ 3.2 ส่วน edge ที่เดินเครือข่ายระหว่างเครื่อง (lateral movement) แบ่งสัดส่วนประมาณ 50% เป็น `cve`, 30% เป็น `misconfig` และ 20% เป็น `deterministic` จำนวน edge ที่แต่ละเครื่องเชื่อม ออกไปได้ (degree) ถูกกำหนดตามระดับการแบ่งส่วนโดยตรง คือ `micro`=1 น้อยกว่า `3tier`=2 น้อยกว่า `flat`=3 เป็นการออกแบบไว้ตั้งแต่ต้น ไม่ใช่ผลพลอยได้จาก การสุ่ม

ส่วน edge ที่ข้ามโดเมนสุ่มมาจากคลาสที่กำหนดไว้ล่วงหน้า 5 ชนิด (เช่น `aws-key-on-host`, `ssrf-imdsv1-token`) จำนวนเส้นที่สุ่มได้ขึ้นกับความ หนาแน่นที่ตั้งไว้และขนาดกราฟ หลังจากสร้างโครงสร้างหลักเสร็จแล้ว อาจมีบางคู่ (entry, target) ที่ยังไปไม่ถึงกัน ตัวซ่อมกราฟ (`_repair`) จึงเติม edge ที่ถูกต้องตามนโยบาย เท่าที่จำเป็นเข้าไป ไม่ใช่ edge สมมติที่แต่งขึ้นมาลอย ๆ เพื่อให้ทุกคู่ไปถึงกันได้แน่นอน การ แบ่งส่วนแบบ `micro` ซึ่ง เข้มงวดกว่าแบบอื่น มักต้องซ่อมมากกว่า นี่คือที่มาของ ข้อสังเกตในหัวข้อ 4.7 ที่ว่าสัดส่วน “ต้องข้าม โดเมน” ที่วัดได้ส่วนหนึ่งมาจาก การซ่อมกราฟ ไม่ใช่โครงสร้างธรรมชาติล้วน ๆ

นอกจากนี้จำนวน entry และ target ยังถูกล็อกไว้ตายตัวตามขนาดกราฟ ($n=50$ ให้ 3 entry กับ 2 target, 200 ให้ $5/3$, 800 ให้ $8/4$) เพราะการคำนวณ `pairwise_dominators` มีต้นทุนที่โตตามจำนวน entry คูณจำนวน target ถ้าไม่ล็อกไว้ ต้นทุนนี้อาจหลุดจากที่ควบคุมได้ง่าย ตารางปัจจัยทั้งหมด (ขนาด × ความหนาแน่น × การแบ่งส่วน) มี 27 เซลล์ แต่ละเซลล์สร้างกราฟซ้ำ 8 ครั้ง (`rep`) รวมเป็น 216 โครงสร้าง เกินขั้นต่ำ 200 กราฟ ตามข้อเสนอแม้นับเฉพาะโครงสร้างก่อนคูณด้วยโมเดลน้ำหนัก seed ของแต่ละโครงสร้างคำนวณ จากตำแหน่งของเซลล์และ `rep` โดยตรง ไม่ขึ้นกับว่าครั้ง นั้นเรียกใช้กี่ขนาดหรือกี่ `rep` ทำให้กราฟ ชุดย่อยใด ๆ ทำซ้ำได้เหมือนเดิมเสมอ (ตรวจสอบ จริงแล้วด้วยการรันข้าม `PYTHONHASHSEED` หลายค่า) และทุกกราฟยังบันทึก `cve_pool_size` กับ `epss_snapshot` กำกับไว้ด้วย เพื่อให้เห็นชัดว่าการทำซ้ำ นี้อ้างอิงข้อมูล snapshot ชุดไหน

2.2.5 นิยามจุดตัด

เมื่อมีกราฟพร้อมน้ำหนักแล้ว คำถามหลักของโครงงานนี้คือ “จุดตัด” (choke point) อยู่ตรง ไหน แต่คำว่าจุดตัดเองมีได้หลายความหมาย โครงงานนี้จึงแยกไว้ชัดเจนเป็นสามนิยาม (รายละเอียด เชิงตัวเลขของแต่ละนิยามอยู่ในหัวข้อ 4.2.1):

- **global** – node ที่อยู่บนทุกเส้นทางของ*ทุกคู่* (entry, target) เป็นนิยาม ที่เข้มที่สุด
- **per-target** – node ที่อยู่บนทุกเส้นทางที่ไปถึง target หนึ่งตัว ไม่ว่าจะมาจาก entry ตัวไหน
- **per-pair** – node ที่อยู่บนทุกเส้นทางของคู่ (entry, target) หนึ่งคู่โดยเฉพาะ เป็นนิยามที่

หลวมที่สุดและให้เซตใหญ่ที่สุด

นิยาม global คำนวณได้สองวิธีที่พิสูจน์ได้ว่าให้ผลเท่ากันเสมอ วิธีแรกเติม super-source เข้าไปทุก entry และ super-sink ออกจากทุก target แล้วใช้ dominator tree มาตรฐาน (ผ่าน `nx.immediate_dominators`) วิธีที่สองคำนวณทีละคู่ (entry, target) แล้วนำ ผลมาตัดฉันทามติ (intersect) กัน เนื่องจากสองวิธีนี้พิสูจน์แล้วว่าเท่ากันเสมอ การรายงาน “อัตราความไม่ตรงกัน” ระหว่างสองวิธีนี้จึงไม่ใช่ผลการทดลอง แต่เป็นเพียงแบบทดสอบความ ถูกต้องของโค้ดเท่านั้น ถ้าตัวเลขไม่ตรงกันจริง แปลว่าโค้ดมีบั๊ก ไม่ใช่ข้อค้นพบ อีกจุดที่ต้อง ระวังคือคู่ (entry, target) ที่ target ไปไม่ถึงเลยต้อง*ข้าม*ไป ไม่นับเป็นเซตว่าง เพราะ ถ้านับเป็นเซตว่าง ความเท่ากันของสองวิธีข้างต้นจะพังทันที เซตว่างตัดกับเซตอะไรก็ได้เซตว่าง เสมอ ซึ่งไม่ใช่สิ่งที่ต้องการวัด

สามนิยามข้างต้นบอกได้แค่ว่า node อยู่ในเซตจุดตัดหรือไม่ แต่ยังจัดอันดับ node ไม่ได้ว่าตัว ไหนสำคัญกว่าตัวไหน เพื่อให้จัดอันดับได้ โครงงานนี้จึงให้คะแนนแต่ละ node ด้วยสูตร $\Delta(v) = \text{cost}(G-v) - \text{cost}(G)$ คือต้นทุนรวมของผู้โจมตีที่เพิ่มขึ้นเมื่อลบ node v ทิ้งไปจากกราฟ (ต้นทุนรวมของผู้โจมตีในที่นี้คือผลรวมต้นทุนของเส้นทางถูกที่สุด ของทุกคู่ที่ยังไปถึงกันได้) ข้อค้นพบที่สำคัญคือ global dominator set พิสูจน์ได้ว่าเท่ากับเซต $\{v : \Delta(v) = \infty\}$ พอดี (ตรวจสอบจริงด้วย brute force ในหัวข้อ 3.4) พูดอีกแบบคือ จุดตัดเชิงโครงสร้างไม่ใช่วิธีที่แยกต่างหากจากคะแนนเชิง ความน่าจะเป็นเลย แต่เป็นกรณีสุดขั้วของมันเอง กรณีที่ต้นทุนเพิ่มขึ้นจนเป็นอนันต์

โค้ดจริงที่คำนวณ $\Delta(v)$ อยู่ในฟังก์ชัน `delta_cost_scores` ของไฟล์ `attackpath/chokepoints.py` ทำตามนิยามนี้ตรงตัวมาก: ลบ node ออกทีละตัว คำนวณต้นทุนรวมใหม่ แล้วลบกับต้นทุนเดิม

```
for v in candidates:
    H = G.copy()
    H.remove_node(v)
    after = _pair_costs(H, entries, targets, weight)
    for pair, c in base.items():
        if pair in after:
            delta += after[pair] - c
        else:
            cut_n += 1
```

สังเกตบรรทัดสุดท้าย: ถ้าลบ `v` แล้วคู่ `(entry, target)` หนึ่งคู่หายไปจาก `after` (คือไปถึงกันไม่ได้อีกเลย) โค้ดจะนับเป็น `cut_n` แทนที่จะพยายามคำนวณ

`after[pair] - c` ซึ่งจะเป็นการลบด้วยอนันต์ที่ไม่มีความหมาย `cut_n` ที่มากกว่า ศูนย์ จึงเทียบเท่ากับ $\Delta(v) = \infty$ ในนิยามทางคณิตศาสตร์พอดี และเป็นเหตุผลที่ฟังก์ชันนี้ คืนค่าเป็นคู่ `(cut_n, delta, blocked)` แทนที่จะคืนตัวเลขเดียว เพื่อให้แยกกรณี “ตัดได้ทั้งคู่” ออกจากกรณี “ทำให้แพงขึ้น” ได้เสมอ ไม่ปนกัน

ข้อควรระวังอีกจุดคือ node ทุกตัวในกลุ่ม $\Delta = \infty$ นี้มีค่าเท่ากันหมด เพราะลบตัวไหน ก็ตัดทุกเส้นทางเหมือนกัน จึงจัดอันดับกันเองภายในกลุ่มไม่ได้เลย ด้วยเหตุนี้ทุกครั้งที่รายงานอันดับจึงต้องกำกับด้วย $n_{\text{inf}} = |\{v : \Delta(v) = \infty\}|$ (จำนวน node ในกลุ่มนี้) เพราะถ้า $n_{\text{inf}} \geq k$ การเลือก top-k จากกลุ่มนี้จะเป็นการเลือกโดยพลการ และค่า Jaccard@k ที่คำนวณได้จะไม่มีความหมายอีกต่อไป ต้องรายงานสัดส่วนกราฟที่เข้าเงื่อนไขนี้ คู่กับ Jaccard@k เสมอ เช่นเดียวกับ $n_{\text{tie_at_}k}$ ที่นับว่า node กี่ตัวมีคะแนน เสมอกับ node อันดับที่ k พอดี (เมื่อคะแนนเท่ากัน จะตัดสินด้วยเลข id แทน เพื่อให้อันดับทำ ซ้ำได้เสมอ)

2.2.6 Weighted Min-Cut

ตามที่กล่าวไว้ในหัวข้อ 2.1.2 โครงงานนี้ต้องการเทียบ dominator กับ weighted min-cut (วัตถุประสงค์ข้อ 3) แต่ปัญหาคือทั้งสองวิธีคืนค่าคนละหน่วยกัน: dominator คืนเซตของ node ส่วน min-cut แบบมาตรฐานคืนเซตของ edge ที่ถูกตัด จะเทียบกันตรง ๆ ไม่ได้ ทางแก้คือเทคนิคที่เรียกว่า node-splitting: แตก node v ทุกตัวออกเป็นสองส่วนคือ v_{in} กับ v_{out} แล้ววางน้ำหนักของ v ไว้บน edge ภายในที่เชื่อมสองส่วน นี้เข้าด้วยกัน ส่วน edge เดิมของกราฟให้ capacity เป็นอนันต์ทั้งหมด เมื่อรัน min-cut มาตรฐาน (`nx.minimum_cut`) บนกราฟที่แตกแล้วนี้ cut ที่ได้จะเลือกตัด edge ภายในเท่านั้น เพราะไม่มีทางถูกกว่าตัด edge เดิมที่ capacity อนันต์เลย ผลลัพธ์จึงแปลงกลับ เป็นเซตของ node ได้ตรง ๆ นอกจากนี้ entry และ target เองก็ถูกกำหนดให้มี capacity อนันต์เสมอด้วย เพราะการ “ป้องกัน” ระบบด้วยการลบจุดเริ่มต้นหรือเป้าหมายทิ้งไปเลยนั้น ไม่ใช่แนวทางป้องกันที่สมเหตุสมผลตั้งแต่แรก

โค้ดส่วนที่ทำหน้าที่แตก node และกำหนด capacity ตามที่อธิบายข้างต้น อยู่ในฟังก์ชัน `node_min_cut` ของไฟล์ `attackpath/chokepoints.py` ใจความสำคัญคือสามบรรทัดนี้:

```
for v in G.nodes:
    c = INF if v in protected else float(cap(G, v))
    S.add_edge((v, "in"), (v, "out"), capacity=c)
for u, v in G.edges:
    S.add_edge((u, "out"), (v, "in"), capacity=INF)
```

บรรทัดแรกคือหัวใจของ node-splitting: สร้าง edge ภายในจาก `(v, "in")` ไป `(v, "out")` โดยให้ capacity เท่ากับน้ำหนักของ node นั้น ยกเว้นถ้า `v` เป็น entry/target (`protected`) จะให้ `INF` แทน (ตัดไม่ได้) บรรทัดที่สองคือการเติม edge เดิมของกราฟกลับเข้าไป แต่บังคับให้ capacity เป็น `INF` เสมอ เพื่อไม่ให้ `nx.minimum_cut` มีทางเลือกตัด edge เดิมได้เลย มันจึงถูกบีบให้ตัดได้เฉพาะ edge ภายในที่เพิ่งสร้างขึ้น ซึ่งแปลงกลับเป็นเซตของ node ได้ตรง ๆ ตามที่อธิบายไว้ข้างต้น

โครงงานนี้รายงานผลของ capacity สามแบบคู่กันเสมอ เพราะแต่ละแบบตอบคำถามต่างกัน แบบแรกคือ `unit` (นับ 1 ต่อ node ไม่สนน้ำหนักเลย) ซึ่งเทียบหน่วยตรงกับ dominator ได้โดยตรง เพราะ dominator เองก็ไม่อ่านน้ำหนักเช่นกัน แบบที่สองคือ `prob` (ใช้ค่า p สูงสุดของ edge ขาเข้า) เป็นค่าตั้งต้นที่ใช้จริง และแบบที่สามคือ `cost` (ใช้ $-\log p$ ของ edge ขาเข้าที่ถูกที่สุด) capacity แบบ `cost` นี้มีปัญหา: บนกราฟที่มี edge เชิง โครงสร้าง ($p = 1$ จึง $w = 0$) node ที่เข้าถึงได้ฟรีจะมี capacity เป็นศูนย์ ทำให้ min-cut ยุบ ไปเลือกเซต capacity ศูนย์ที่เลือกได้โดยพลการ เพราะตัดตรงไหนก็ต้นทุนเท่ากันคือศูนย์ วัด ได้จริงในหัวข้อ 4.2.3 เพราะเหตุนี้ `cost` จึงรายงานไว้เป็นเพียง คอลัมน์ sensitivity เสริม ไม่ใช่ค่าหลักของการเปรียบเทียบ

2.2.7 การออกแบบการทดลอง

เมื่อมีทั้งกราฟ น้ำหนัก และวิธีวัดจุดตัดครบแล้ว ส่วนสุดท้ายของการออกแบบคือการวางแผนว่า จะทดลองอะไรบ้าง หลักการสำคัญคือการทดลองทั้งหมดใช้กราฟโครงสร้างเดียวกันทุกครั้งที่ เป็นไปได้ คือสร้างโครงสร้างกราฟขึ้นมาครั้งเดียว แล้วค่อยถ่วงน้ำหนักใหม่ด้วย 4 โมเดลที่ ต่างกัน เพื่อให้การเปรียบเทียบเป็นแบบจับคู่ (paired comparison) คือเทียบผลบนกราฟ โครงสร้างเดียวกันเป๊ะ ไม่ใช่เทียบกราฟคนละชุดซึ่งจะทำให้ตีความความต่างยากขึ้นมาก ตารางที่ 1 สรุปการทดลองทั้งเจ็ดชุดและคำถามที่แต่ละชุดตอบ

ตารางที่ 1 การทดลองทั้งเจ็ดชุด

exp	สิ่งที่วัด
1a	dominator set เทียบกับ weighted node min-cut (สามสเกล capacity) บนกราฟเดียวกัน
1b	สามนิยามของจุดตัด (global/per-target/per-pair) และสัดส่วนที่แต่ละนิยามไม่ว่าง
2	อันดับเปลี่ยนเท่าไรเมื่อรบกวนน้ำหนักแบบสมมาตร (± 10–50%) และแบบเชิงประจักษ์ (สเกล $0.5\times$–$8\times$ ของ residual จริงจากการ fit)
3	อันดับเปลี่ยนเท่าไรเมื่อสุ่มลบ edge (5–20%)
4	เปรียบเทียบ 4 โมเดลน้ำหนัก (epss/cvss/uniform/random) แบบจับคู่บนโครงสร้างเดียวกัน
6	อันดับเปลี่ยนเท่าไรเมื่อใช้ fitted p แทน EPSS จริงบน CVE edge เดียวกัน
7	สัดส่วนของเส้นทางถูกสุดที่ใช้ edge ข้ามโดเมน แยกตาม domain ของ target

ที่ในตารางไม่มี exp 5 เพราะคำถามของ exp 5 (จุดตัดเปลี่ยนเท่าไรตามการแบ่งส่วนเครือข่าย) ไม่ต้องรันกราฟเพิ่มเลย การแบ่งส่วนเป็นปัจจัยหนึ่งที่มีอยู่แล้วใน exp 1b จึงวิเคราะห์จากแถว ของ exp 1b โดยตรงได้ทันที (ผลอยู่ในหัวข้อ 4.2.4)

2.2.8 ตัวชี้วัด

สุดท้ายคือคำถามว่าจะอ่านผลการทดลองด้วยตัวเลขอะไร ตัวชี้วัดหลักที่ใช้ตลอดทั้งรายงาน คือ **Jaccard@**k ($k = 5, 10$) เทียบว่า node ที่ติดอันดับ top-k ก่อนและหลังการรบกวน (เช่น รบกวนน้ำหนักหรือลบ edge ออก) ทับซ้อนกันมากน้อยแค่ไหน คิดจากสูตรง่าย ๆ คือขนาด ของเซตที่ node ทั้งสองชุดมีร่วมกัน หารด้วยขนาดของเซตรวมทั้งสองชุด ค่า 1.0 แปลว่า top-k เหมือนเดิมทุกตัว ส่วน 0 แปลว่าไม่เหลือ node ร่วมกันเลย ตัวอย่างเช่น ถ้า top-5 ก่อนรบกวนคือ $\{A, B, C, D, E\}$ และหลังรบกวนกลายเป็น $\{A, B, C, D, F\}$ (มี F เข้ามา แทน E) เซตร่วมมี 4 ตัว (A, B, C, D) ส่วนเซตรวมมี 6 ตัว (A, B, C, D, E, F) Jaccard@5 จึงเท่ากับ $4/6 \approx 0.667$

ตัวชี้วัดรองคือ Kendall's τ วัดว่าลำดับคู่ node สองตัวสลับที่กันหรือไม่เมื่อเทียบก่อน–หลัง คิดเป็นสัดส่วนของคู่ที่ลำดับคงเดิมลบด้วยคู่ที่ลำดับสลับ ค่าอยู่ระหว่าง -1 ถึง 1 เป็นตัวชี้วัดรองเพราะเมื่อคำนวณบนทั้งกราฟ มักถูก tie จำนวนมากกลบ node ส่วนใหญ่ที่ไม่อยู่บนเส้นทาง ถูกสุดเลยจะมีคะแนนเท่ากันหมดคือศูนย์ ทำให้ τ วัดอะไรไม่ค่อยได้ ตัวชี้วัดทั้งสองคำนวณ จากคะแนน

$\Delta(v)$ ที่นิยามไว้ในหัวข้อ 2.2.5 เท่านั้น

นอกจากตัวชี้วัดด้านอันดับจุดตัดสองตัวข้างต้น หัวข้อ 4.1 ที่ประเมินความแม่น ของฟังก์ชันแปลง CVSS⟶EPSS ยังใช้ Spearman's ρ กำกับคู่กับ R^2 เสมอ ρ คือสัมประสิทธิ์สหสัมพันธ์แบบอันดับ (rank correlation) แทนที่จะเทียบค่าตัวเลขจริง แบบที่ R^2 ทำ มันแปลงข้อมูลทั้งสองชุดเป็น*อันดับ*ก่อน แล้ววัดว่าอันดับของสองชุด สอดคล้องกันแค่ไหน ค่าเข้าใกล้ 1 แปลว่าเรียงลำดับตรงกันเกือบสมบูรณ์ รายงานคู่กับ R^2 เสมอเพราะ R^2 ไวต่อค่าตัวเลขที่คลาดเคลื่อนไปมาก (เช่น ทำนายผิดสิบเท่า) แต่ ρ สนใจแค่ว่าอันดับถูกหรือไม่ ซึ่งตรงกับสิ่งที่โครงงานนี้ต้องการจริง ๆ คือการจัดอันดับจุดตัด ไม่ใช่ การทำนายค่าตัวเลขให้แม่น

Jaccard@k ยังมีสองรูปแบบที่รายงานคู่กันเสมอ แบบแรกคือแบบปกติ (ตัด top-k ตรง ๆ ตามลำดับ) แบบที่สองนับ node ที่คะแนนเสมอกับ node อันดับที่ k เข้าไปด้วย (`top_k_set`) ต้องมีแบบที่สองเพราะโครงสร้างแบบโซ่ของกราฟทำให้ node ที่ติดกัน หลายตัวมีคะแนนเท่ากันเป๊ะได้บ่อย ถ้าใช้ plain top-k อย่างเดียว การตัดกลุ่มที่คะแนน เท่ากันตามเลข id จะให้ค่า Jaccard สูงเกินจริง เพราะวิธีตัดสินใจ (tie-break) เหมือนกันทุก ครั้งอยู่แล้ว

ในการรายงานตัวเลขทั้งหมด ค่าเฉลี่ยทุกตัวคำนวณบน*กราฟ* ไม่ใช่บนแถวของผลลัพธ์ เพราะแต่ละกราฟมีการทำซ้ำ (`rep`) ของสัญญาณรบกวนหลายครั้ง แถวพวกนั้นไม่เป็น อิสระต่อกัน มาจากกราฟเดียวกัน ถ้านับทุกแถวเท่ากันจะประเมินความแม่นยำสูงเกินจริง ช่วง ความเชื่อมั่น 95% ของค่าเฉลี่ยจึงคำนวณด้วยวิธี bootstrap แบบสุ่มเลือก*กราฟ*ซ้ำได้ (resample with replacement) ไม่ใช่สุ่มเลือกแถว ส่วนสัดส่วนของกราฟที่มีคุณสมบัติหนึ่ง ๆ (เช่น "มี global choke point") ใช้ช่วงความเชื่อมั่นแบบ Wilson แทน เพราะจำนวนกราฟที่ เข้าเงื่อนไขแบบนี้มักมีน้อย ทำให้สูตรปกติ (normal approximation) ให้ช่วงที่ผิดสัดส่วนได้ง่าย

บทที่ 3 เนื้องาน

บทนี้อธิบายว่าโมเดลและการออกแบบในบทที่ 2 ถูกเขียนเป็นโค้ดจริงอย่างไร เริ่มจากภาพรวมของโครงสร้างโปรเจกต์ ไปจนถึงรายละเอียดที่สำคัญที่สุดสามเรื่องคือ การทำ ให้ตัววิเคราะห์ไม่ผูกติดกับ Neo4j (หัวข้อ 3.3) การทำให้กราฟทุกใบทำ ซ้ำได้จริง (หัวข้อ 3.4) และวิธีสร้าง lab จริงขึ้นมา (หัวข้อ 3.5)

3.1 โครงสร้างโปรเจกต์

โค้ดทั้งหมดเขียนด้วย Python 3.13 บริหารจัดการ dependency ด้วย `uv` และแบ่งเป็น ห้าส่วน ตามหน้าที่ ดังนี้

- `attackpath/` – แพ็กเกจหลักของโครงงาน ประกอบด้วยโมเดลกราฟ (`graph.py`), ตัวสร้างกราฟจำลอง (`gen.py`), การให้น้ำหนัก (`weights.py`), ตัววิเคราะห์จุดตัด (`chokepoints.py`), การทดสอบความ ทนทาน (`robustness.py`) และสคีมาผลลัพธ์ (`results.py`) แพ็กเกจนี้ไม่พึ่ง Neo4j เลยแม้แต่บรรทัดเดียว เหตุผลอยู่ในหัวข้อ 3.3
- `experiments/` – ตัวขับเคลื่อนการทดลองทั้งเจ็ดชุด (`run.py`) พร้อมตัว วิเคราะห์ผลเป็นตารางและกราฟ (`analyze.py`) และ exp 6 ซึ่งแยกสคริปต์ต่างหาก
- `scripts/` – ท่อประมวลผลข้อมูลและตัวตรวจสอบ `pull_data.py` ดึง EPSS กับ CVSS มา join กัน (รายละเอียดในหัวข้อ 3.2) `fit_weights.py` ฝึกฟังก์ชัน CVSS⟶EPSS ทั้ง 4 แบบแล้วเลือกโมเดลที่ใช้จริง `build_vectors.py` กับ `calibrate_rubric.py` ตรวจและนับความครบถ้วน ของ rubric misconfiguration (ภาคผนวก 5.3) `audit_data.py` ตรวจสอบการ join ข้อมูลซ้ำด้วยโค้ดคนละชุดจากที่ใช้จริง ส่วน `check_claims.py` อ่านตัวเลขทุกตัวที่ปรากฏในเอกสารฉบับนี้ แล้วคำนวณใหม่จาก `results/` กับ `data/fit/` ว่าตรงกันทุกตัวหรือไม่ ถ้าไม่ตรงจะแจ้งชื่อไฟล์และตัวเลขที่ผิดทันที
- `lab/` – ไฟล์ Terraform, docker-compose ของฝั่ง on-prem และตัว export จาก Neo4j
- `tests/` – ชุดทดสอบ 70 ไฟล์ย่อย (รายละเอียดในหัวข้อ 3.6)

3.2 ท่อประมวลผลข้อมูลและการ fit

ก่อนจะ fit ฟังก์ชัน CVSS⟶EPSS ได้ ต้องมีข้อมูลที่มีทั้งสองค่าคู่กันก่อน ขั้นตอนนี้ทำโดย `scripts/pull_data.py` ซึ่งดึงข้อมูลสองแหล่งแล้วนำมา join กัน แหล่งแรกคือคะแนน EPSS ทั้งหมด ณ วันที่กำหนด (ไฟล์ snapshot จาก `epss.empiricalsecurity.com`) แหล่งที่สองคือเวกเตอร์ CVSS v3.1/v3.0 จาก NVD API 2.0 ซึ่งดึงเป็นบล็อกละ 2,000 รายการ และแคชทุกบล็อกลงดิสก์ถาวร เพื่อให้รันซ้ำได้โดยไม่ต้องดึงใหม่ทุกครั้ง

ผลจากการ join ได้ CVE ที่มีทั้งเวกเตอร์และ EPSS พร้อมกัน 46,543 รายการ จากนั้นตัด CVE ที่เผยแพร่ก่อนปี 2016 ออกไป เพราะปีนั้นคือปีที่ CVSS v3 เริ่มใช้งานจริงอย่างเป็นระบบ เหลือ 46,360 รายการที่เข้า fit จริงผ่าน `scripts/fit_weights.py` โมเดลที่ใช้ตั้ง ราคา edge จริงคือการถดถอยเชิงเส้นจาก 8 ตัวแปรของ CVSS (AV/AC/PR/UI/S/C/I/A แบบ one-hot) ในสเกล logit ของ EPSS เลือกใช้รูปแบบนี้เพราะให้ค่าสัมประสิทธิ์ที่ตรวจสอบ ย้อนกลับได้ทีละตัวแปร ตรงกับที่ต้องการในวัตถุประสงค์ข้อ 2

3.3 ตัววิเคราะห์จุดตัด

ตัววิเคราะห์ (`chokepoints.py`, `robustness.py`) อ่านกราฟจากไฟล์ edge list ธรรมดาเท่านั้น ไม่ว่าจะเป็น JSON หรือ CSV และไม่เรียก Neo4j driver เลยแม้แต่ บรรทัดเดียว ทำแบบนี้เพราะวัตถุประสงค์ข้อ 5 (หัวข้อ 1.3) บังคับ ให้ต้องพิสูจน์ว่าโมเดลใช้กับระบบจริงได้ ไม่ใช่ใช้ได้เฉพาะข้อมูลที่ออกแบบมาเอง กราฟจำลอง กับกราฟจาก lab จริงจึงต้องไหลผ่านโค้ดวิเคราะห์ตัวเดียวกันได้โดยไม่ต้องแก้โค้ดสักบรรทัด การเชื่อมต่อกับ Neo4j ทั้งหมดจึงถูกจำกัดไว้ในเฟส lab เท่านั้น นั่นคือ `lab/exporter/export_graph.py` (หัวข้อ 3.5) ไฟล์นี้ทำหน้าที่ เดียวคือแปลงผลสืบค้น Cypher จาก Cartography ให้กลายเป็น edge list สคีมาเดียวกับ ที่ตัวสร้างกราฟจำลองสร้างขึ้นมา

3.4 การทำซ้ำผลได้ (Reproducibility)

กราฟจำลองทุกกราฟต้องทำซ้ำได้จาก seed เดียวกันเป๊ะ ฟังดูเป็นเรื่องง่าย แต่ระหว่างการ พัฒนาจริงกลับเจอปัญหาที่ทำให้ทำซ้ำไม่ได้ถึงสามจุด ทั้งสามจุดนี้แก้ไขหมดแล้ว:

1. **การเรียงลำดับของ `set` ไม่แน่นอน** ขั้นตอนซ่อมกราฟขั้นสุดท้าย (ใช้ เมื่อไม่มีทางซ่อมอื่นเหลือ) เคยเลือก node ปลายทางด้วย `next(iter(...))` บน ออบเจกต์ `set` ของ Python ปัญหาคือลำดับของ `set` ขึ้นกับการสุ่มแฮชสตริง ของ Python ต่อโปรเซส (**PYTHONHASHSEED**) ทำให้กราฟที่ใช้ seed เดียวกันกลับ ต่างกันได้ระหว่างสองรัน แก้ไข

โดยเรียก `sorted()` ก่อนเลือกเสมอ:

```
# before: order of co depends on PYTHONHASHSEED
dst = next(x for x in co if x != e)

# after: sorted() fixes the order every time
dst = next(x for x in sorted(co) if x != e)
```

2. **`hash()`** การถ่วงน้ำหนักแต่ละโมเดลเคยใช้ `seed + hash(model) % 1000` เพื่อให้แต่ละโมเดลน้ำหนักได้ seed ย่อยที่ต่างกัน แต่ `hash()` ของสตริงใน Python สุ่มต่อโปรเซสเช่นเดียวกับปัญหาข้อแรก แก้ไขโดยเปลี่ยนไปใช้ `zlib.crc32()` ซึ่งให้ค่าคงที่เสมอไม่ว่าจะรันกี่ครั้ง
3. **ไฟล์ผลลัพธ์ถูกต่อท้าย (append) โดยไม่ตั้งใจ** `Writer` เปิดไฟล์ `results/exp*.csv` แบบ append เพื่อให้เขียนหัวตารางเพียงครั้งเดียว แต่เมื่อรันกริด ใหม่ทับของเก่าที่ยังไม่ได้ลบไฟล์ แถวเก่ากับแถวใหม่จะปนกัน (เกิดขึ้นจริงสองครั้งระหว่างการ พัฒนา) แก้ไขโดยให้ตรวจก่อนรันว่ามีไฟล์เดิมอยู่หรือไม่ ถ้ามีจะหยุดทันทีและต้องใช้ `--force` เพื่อลบก่อนรันใหม่เท่านั้น

ทั้งสามจุดนี้ตรวจพบด้วยวิธีเดียวกัน คือรันตัวสร้างกราฟหรือตัวทดลองซ้ำข้ามค่า `PYTHONHASHSEED` หลายค่า แล้วเทียบไฟล์ผลลัพธ์ว่าเหมือนกันทุกบิตหรือไม่ วิธีนี้ กลายเป็นขั้นตอนมาตรฐานก่อนรันกริดเต็มทุกครั้งหลังจากนี้ นอกจากนี้ `git_rev` ที่ บันทึกไว้ในทุกแถวผลลัพธ์ยังต่อท้ายด้วย `+dirty` เมื่อโค้ดที่ผลิตผลนั้นมีการแก้ไขที่ยัง ไม่ได้ `commit` เพื่อป้องกันไม่ให้แถวผลลัพธ์อ้างว่ามาจาก commit ที่จริง ๆ แล้วไม่ได้ ผลิตมันขึ้นมา

3.5 Lab: Terraform, Cartography, Neo4j

ฝั่งคลาวด์ของ lab ใช้ Terraform สร้างทรัพยากรจริงบน LocalStack เวอร์ชัน community ซึ่งไม่ต้องมีบัญชี AWS จริง (เหตุผลเต็มอยู่ในภาคผนวก 5.3) แล้วรัน Cartography เพื่อดึงทรัพยากรที่สร้างนั้นเข้า Neo4j `lab/run_sync.py` ครอบ CLI ของ Cartography ไว้อีกชั้นหนึ่ง เพียงเพื่อ monkeypatch ฟังก์ชันเดียวคือ `iam.sync_service_last_accessed_details` ที่ LocalStack community ไม่ รองรับและทำให้ทั้งขั้นตอนค้างอยู่กับที่ การทำแบบนี้ไม่ใช่การเขียน collector ขึ้นใหม่แทน Cartography (เรื่องนั้นอยู่นอกขอบเขตของโครงงานตามที่ระบุไว้ในหัวข้อ 1.4) แต่เป็นการแก้จุดบกพร่องเล็ก ๆ จุดเดียวเท่านั้น

ฝั่ง on-prem จำลองด้วย `docker-compose` สองเครื่องบนเครือข่าย flat เดียวกัน การไม่แบ่งเครือข่ายแบบนี้*คือ*ตัว misconfig `flat-vlan-no-segmentation` เอง โดยตรง ไม่ใช่ทางลัดข้ามการจำลองมัน

ก่อนจะปล่อยให้ edge `aws-key-on-host` ปรากฏในกราฟ `lab/exporter/onprem.py` ต้องยืนยันขั้นตอนจริงสองอย่างก่อนเสมอ อย่างแรกคือ สแกนด้วย `nmap` (เครื่องมือ สแกนพอร์ตเครือข่ายมาตรฐาน บอกได้แค่ว่าเครื่องเป้าหมายเปิดบริการอะไรอยู่บ้าง ไม่บอก ว่ามีช่องโหว่อะไร) ว่าทั้งสองโฮสต์เปิดพอร์ต SSH จริง อย่างที่สองคือลองเจาะรหัสผ่าน root ด้วย dictionary attack จริงผ่าน `paramiko` ไลบรารี Python สำหรับสั่งงาน SSH ด้วยโค้ด ใช้แทนการพิมพ์คำสั่ง `ssh` เอง การเจาะแบบนี้คือการลองรหัสผ่านที่พบบ่อย ในฐานข้อมูลรหัสผ่านที่เคยหลุดจริงทีละตัว จนกว่าจะเข้าได้หรือหมดรายการ ถ้าเจาะไม่สำเร็จ จะไม่มี edge ให้สิทธิ์ผ่านมาเลย เพื่อป้องกันไม่ให้กราฟกระโดดข้ามขั้นที่ผู้โจมตีต้องมี shell จริงก่อน

edge ข้ามโดเมนเส้นเดียวของกราฟ lab (`aws-key-on-host`) ต้องมีหลักฐานจาก *สองแหล่งอิสระ*ที่ตรงกันด้วย: รหัส access key ที่อ่านได้จากไฟล์บนคอนเทนเนอร์ on-prem จริง ต้องตรงกับ `AccountAccessKey` node ที่ Cartography ค้นพบ เองจากฝั่งคลาวด์ หากไม่ตรงกัน `export_graph.py` จะปฏิเสธสร้าง edge นั้นทันที ข้อนี้ทดสอบจริงแล้วด้วยการแก้ไขรหัสบนโฮสต์ให้ไม่ตรงกับฝั่งคลาวด์โดยตั้งใจ (ดูหัวข้อ 4.8) การออกแบบแบบนี้ทำให้ edge ข้ามโดเมนเป็นการค้นพบที่เกิดจากข้อมูล จริงสองฝั่งเห็นพ้องต้องกัน ไม่ใช่ข้อเท็จจริงที่ปลูกไว้ฝ่ายเดียวแล้วอ้างว่าเป็นการค้นพบ

3.6 การทดสอบซอฟต์แวร์

ชุดทดสอบทั้งหมดมี 2,171 กรณี และผ่านทั้งหมด แบ่งเป็นสองชั้นที่ตรวจคนละเรื่อง ชั้นแรก คือ เทสต์หน่วยปกติ (`test_gen.py`, `test_dominator.py`, `test_nodecut.py`, `test_weights.py`, `test_robustness.py`, `test_cross_share.py`) ซึ่งตรวจคุณสมบัติเฉพาะจุดของแต่ละฟังก์ชัน เช่น ความ เท่ากันของ post-dominator กับ pairwise-dominator, min-cut ต้องคืนเซต node ไม่ใช่ edge, คู่ที่ไปไม่ถึงกันต้องถูกข้ามไม่ใช่นับเป็นเซตว่าง

ชั้นที่สองคือ `test_bruteforce.py` ซึ่งตรวจในอีกระดับหนึ่ง แทนที่จะตรวจคุณสมบัติ เฉพาะจุดแบบชั้นแรก มันเทียบผลของตัววิเคราะห์กับคำตอบที่คำนวณขึ้นใหม่จากนิยามตรง ๆ โดยไม่เรียกโค้ดที่กำลังตรวจสอบเลย ทำบนกราฟสุ่มขนาดเล็ก 300 กราฟ ครอบคลุมทั้งสามนิยามของ dominator ความเหมาะสมที่สุดของ weighted node min-cut ทั้งสามสเกล capacity

เทียบกับการไล่ทุกเซตย่อยของ node และคะแนน $\Delta(v)$ ที่คำนวณใหม่ด้วย Dijkstra ล้วน ๆ ทีละ node

เพื่อให้มั่นใจว่าชุดทดสอบนี้ตรวจจับบั๊กได้จริง ไม่ใช่แค่ผ่านไปเฉย ๆ จึงลองฉีดบั๊กจริงสามแบบ เข้าไปในโค้ด ได้แก่ตัวกรอง candidate ที่ทิ้ง node สำคัญไป, min-cut ที่ไม่สนใจ capacity ที่ระบุ และการนับคู่ที่ไปไม่ถึงกันเป็นเซตว่าง ผลคือชุดทดสอบจับผิดได้ครบทั้งสามแบบ จึงค่อย นำบั๊กที่ฉีดเข้าไปออก

บทที่ 4 ประเมินผล อภิปราย และวิเคราะห์

บทนี้รายงานผลการทดลองทั้งเจ็ดชุดที่วางแผนไว้ในหัวข้อ 2.2.7 เรียง ตามวัตถุประสงค์ของข้อเสนอโครงงาน เริ่มจากความแม่นของการให้น้ำหนัก (วัตถุประสงค์ข้อ 2) ไปจนถึงผลจาก lab จริง (วัตถุประสงค์ข้อ 5) แล้วปิดท้ายด้วยการอภิปรายว่าผลเหล่านี้มีจุดอ่อน ตรงไหนบ้าง (หัวข้อ 4.9 และ 4.10) ตัวเลขทุกตัวในบทนี้ คำนวณจากไฟล์ผลลัพธ์ดิบใน `results/*.csv` ที่เกิดจากการรัน `experiments/run.py` บนกริด 216 โครงสร้าง × 4 โมเดลน้ำหนัก (รวม 864 กราฟถ่วงน้ำหนัก) แล้วสรุปเป็น ตารางด้วย `experiments/analyze.py` (ขั้นตอนทำซ้ำเต็มอยู่ในภาคผนวก 5.3) และทุกครั้งที่แก้ตัวเลขในบทนี้ จะตรวจสอบย้อนกลับด้วย `scripts/check_claims.py` ว่ายังตรงกับไฟล์ผลลัพธ์จริงเสมอ ไม่ใช่ตัวเลขที่พิมพ์ลอย ๆ โดยไม่มีไฟล์รองรับ

4.1 ผลการประมาณค่า $p(e)$ จาก CVSS (วัตถุประสงค์ข้อ 2)

ในหัวข้อ 2.2.2 อธิบายไว้ว่า edge ที่เป็น misconfiguration ไม่มี CVE ผูกอยู่ จึงไม่มีคะแนน EPSS ให้ใช้โดยตรง ทางแก้ของโครงงานนี้คือกำหนดเวกเตอร์ CVSS ให้แต่ละ คลาสของ misconfiguration ตาม rubric (ภาคผนวก 5.3) แล้วใช้ฟังก์ชันที่ เรียนรู้ความสัมพันธ์ระหว่างเวกเตอร์ CVSS กับคะแนน EPSS ของ CVE จริง แปลงเวกเตอร์นั้น เป็นค่าความน่าจะเป็น $p(e)$ หัวข้อนี้ตอบคำถามว่า ฟังก์ชันที่แปลงให้นั้นแม่นยำแค่ไหน ซึ่ง สำคัญเพราะความคลาดเคลื่อนของ $p(e)$ ส่งผลต่ออันดับเส้นทางโดยตรง

4.1.1 ข้อมูลและวิธีการ

ข้อมูลที่ใช้ฝึกฟังก์ชันนี้คือ CVE จำนวน 46,360 รายการ ที่มีทั้งเวกเตอร์ CVSS และคะแนน EPSS ครบทั้งคู่ เวกเตอร์ CVSS ดึงมาจาก NVD (National Vulnerability Database ฐานข้อมูล ช่องโหว่กลางของสหรัฐฯ) ผ่าน API เวอร์ชัน 2.0 เป็นบล็อกละ 2,000 รายการ รวม 33 หน้า ด้วยสคริปต์ `scripts/pull_data.py` ตามที่อธิบายไว้ในหัวข้อ 3.2 ส่วนคะแนน EPSS มาจาก snap-shot วันที่ 2026-08-26 ไฟล์ดิบดาวน์โหลดจาก `epss.empiricalsecurity.com` เว็บทางการที่ FIRST.org เผยแพร่คะแนน EPSS รายวัน เป็น CVE ที่เผยแพร่ระหว่าง 2017-03-20 ถึง 2026-08-06 การสุ่มทุกขั้นตอนใช้ seed เดียวกันคือ 20260826 เพื่อให้ทำซ้ำได้ ตัวแปรตามที่ใช้ฝึกคือ EPSS ในสเกล logit คือ $\log \frac{p}{1-p}$ (แปลงความน่าจะเป็นที่ปกติอยู่ในช่วง 0–1 ให้กระจายไปทั่วเส้นจำนวนจริง แทนที่จะกระจุก ตัวอยู่ใกล้ 0 อย่างที่ค่า EPSS ส่วนใหญ่เป็นจริง) ไม่ใช่สเกล

ความน่าจะเป็นตรง ๆ เหตุผลที่ เลือกฝึกในสเกลนี้คือค่าความน่าจะเป็นดิบของ EPSS เบ้ไปทางค่าน้อยมาก (ส่วนใหญ่ต่ำกว่า 1%) การถดถอยเชิงเส้นตรง ๆ บนสเกลดิบจะให้น้ำหนักกับ CVE ที่หายากแต่ค่าสูงน้อยเกินไป และเปรียบเทียบฟังก์ชัน 4 แบบด้วยกัน ได้แก่ ค่าเฉลี่ยคงที่ (constant baseline) เป็นเกณฑ์ขั้น ต่ำ, การถดถอยเชิงเส้นจากคะแนนฐาน CVSS เพียงตัวเดียว, การถดถอยเชิงเส้นจาก 8 metric ของ CVSS เรียกย่อว่า OLS (Ordinary Least Squares) ชื่อเรียกการถดถอยเชิงเส้นแบบ “กำลังสองน้อยที่สุด” ในหมู่งานสถิติ หมายถึงวิธีหาสัมประสิทธิ์ที่ทำให้ผลรวมของค่าคลาดเคลื่อน ยกกำลังสองน้อยที่สุด เป็นวิธีถดถอยเชิงเส้นมาตรฐานที่สุด และสุดท้ายคือ gradient boosting จาก 8 metric เดียวกัน เรียกย่อว่า GBM โมเดลนี้คือ ensemble ของ decision tree หลายร้อยต้น ที่เรียนแก้ไขความผิดพลาดของกันและกันทีละขั้น ให้ความแม่นสูงกว่า OLS ได้ แต่ไม่สามารถ อ่านได้ว่าแต่ละตัวแปรมีผลต่อผลลัพธ์อย่างไร ต่างจาก OLS ที่บวกสัมประสิทธิ์ทีละตัวแปรตรง ๆ ตัวชี้วัด ความแม่นในตารางที่ 2 คำนวณบนข้อมูลทดสอบ 25% (สุ่มแบ่งจาก CVE ทั้ง 46,360 รายการ ด้วย seed เดียวกันคือ 20260826) ที่กันไว้ไม่ให้ฟังก์ชันเห็นตอนฝึก

4.1.2 ผลการ fit

ตัวเลขหลักที่ใช้อ่านตารางถัดไปคือ R^2 (สัมประสิทธิ์การตัดสินใจ) ซึ่งตอบคำถามว่า “โมเดลอธิบายความแปรปรวนของข้อมูลจริงได้กี่เปอร์เซ็นต์” ค่า 1.0 แปลว่าทำนายตรงเป๊ะทุก จุด ค่า 0 แปลว่าทำนายได้พอ ๆ กับการทายค่าเฉลี่ยคงที่ลอย ๆ และค่าติดลบแปลว่าแย่กว่า การทายค่าเฉลี่ยเสียอีก ตารางรายงาน R^2 สามสเกลต่างกัน (logit, prob, top 10%) เพราะ ตัวเลขเดียวกันให้ความรู้สึกต่างกันมากตามสเกลที่วัด ดังจะเห็นในสามข้อถัดไป

ตารางที่ 2 ผลการ fit จากเวกเตอร์ CVSS ไปเป็น EPSS (ข้อมูลทดสอบ 25%)

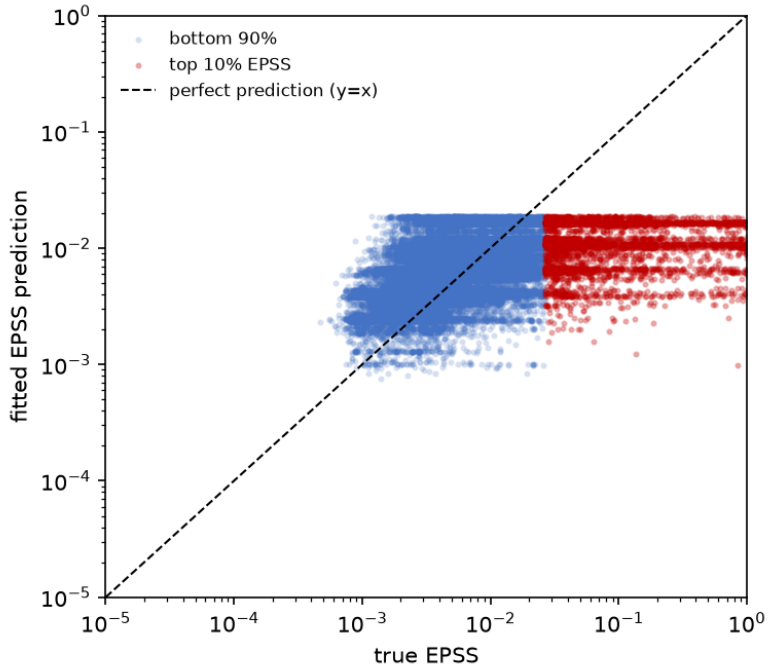
โมเดล	R^2 (logit)	R^2 (prob)	R^2 (top 10%)	ρ
constant baseline	0.000	-0.032	-0.452	—
OLS จากคะแนนฐานตัวเดียว	0.118	-0.018	-0.432	0.354
OLS จาก 8 metric (ใช้จริง)	**0.219**	-0.010	-0.420	0.548
GBM จาก 8 metric (ไม่ได้ใช้)	0.268	-0.005	-0.415	0.601

ตารางที่ 2 บอกสามเรื่องที่ต้องอ่านด้วยกัน

1. **โมเดลที่ใช้ตั้งราคา edge จริงคือ OLS จาก 8 metric ไม่ใช่ตัวที่ได้คะแนนสูงสุด** GBM ให้ค่าสูงกว่าเล็กน้อย แต่เป็นกล่องดำที่อธิบายไม่ได้ว่าทำไมจึงให้ค่านี้ ขณะที่ OLS บวก สัมประสิทธิ์แต่ละตัวกันในสเกล logit ตรวจสอบย้อนกลับได้ทีละ metric ซึ่งตรงกับวัตถุประสงค์ ข้อ 2 ที่ว่า $p(e)$ ต้องอธิบายที่มาได้ ตัวเลขทั้งหมดที่รายงานต่อจากนี้จึงเป็นของ OLS เสมอ

2. **ในหน่วยที่ใช้งานจริง ฟังก์ชันนี้ไม่ดีกว่าการทายค่าเฉลี่ยเลย** R^2 ใน สเกลความน่าจะเป็น (คอลัมน์ “prob”) ติดลบทุกโมเดล รวมถึง OLS ด้วย ค่า 0.219 ที่ดูเหมือน พอใช้ได้จึงเป็นเพียงตัวเลขในสเกล logit เท่านั้น ซึ่งอธิบายความแปรปรวนได้ประมาณหนึ่งใน ห้าของข้อมูล
3. **ความคลาดเคลื่อนไม่ได้กระจายสม่ำเสมอ แต่กระจุกอยู่ที่ EPSS สูง** ฟังก์ชันที่ fit ได้ทำนายค่าสูงสุดเพียง 0.0186 ขณะที่ EPSS จริงสูงสุดถึง 0.99999 พูดง่าย ๆ คือฟังก์ชันนี้ “ไปไม่ถึงหาง” ของการแจกแจง จึงถดถอยเข้าหาค่าฐาน (base rate) ที่ประมาณ 0.0055 แทน

รูปที่ 2 แสดงข้อสามข้างต้นให้เห็นเป็นภาพ: แกนนอนคือ EPSS จริง แกนตั้ง คือค่าที่ฟังก์ชัน fit ทำนาย (ทั้งสองแกนเป็น log scale) ถ้าฟังก์ชันทำนายแม่น จุดทั้งหมดควร เกาะอยู่บนเส้นประทแยงมุม (y=x) แต่สิ่งที่เห็นจริงคือจุดสีแดง (CVE ที่มี EPSS สูงสุด 10%) แทบทั้งหมดอยู่*ใต้* เส้นทแยงมุมมาก และค่าที่ทำนายได้ไม่เคยพุ่งขึ้นไปเกินแนวนอนแคบ ๆ แถวประมาณ 0.01–0.02 เลย ไม่ว่า EPSS จริงจะสูงแค่ไหนก็ตาม ภาพนี้คือ “ฟังก์ชันไปไม่ ถึงหาง” ที่กล่าวไว้ข้างต้น เห็นเป็นรูปธรรมชัดเจน ไม่ใช่แค่ตัวเลขลอย ๆ



รูปที่ 2 EPSS จริงเทียบกับค่าที่ฟังก์ชัน fit ทำนาย (log–log scale, CVE 46,360 รายการ) จุดสีแดงคือ CVE ที่มี EPSS จริงอยู่ใน 10% บนสุด

อีกจุดที่ต้องระวังคือค่า $R^2 = 0.219$ ในตารางที่ 2 มาจากการแบ่งข้อมูลทดสอบแบบสุ่มราย CVE ซึ่งมองโลกในแง่ดีกว่าการใช้งานจริงเล็กน้อย เพราะ CVE ที่คล้ายกันมากอาจ ตกอยู่คนละฝั่งของการแบ่ง ทำให้ฟังก์ชัน “จำ” รูปแบบที่เห็นจากฝั่งฝึกได้บางส่วน เพื่อตรวจสอบ

เรื่องนี้ จึงลองแบ่งข้อมูลแบบที่เข้มงวดขึ้น: เมื่อกันทั้งหน้า NVD ออกจากกันแทนที่จะสุ่มราย CVE R^2 ลดเหลือ 0.156 และเมื่อกันทั้งปีออกจากกัน R^2 ลดเหลือ 0.150 กรณีที่เข้มที่สุดคือฝึก ด้วย CVE ถึงปี 2022 เท่านั้น แล้วทดสอบกับ CVE ตั้งแต่ปี 2023 เป็นต้นไป กรณีนี้ R^2 ติดลบไปเลย (-0.863) แต่ที่น่าสนใจคือ Spearman ρ ยังอยู่ที่ 0.542 แทบไม่ตก ผล นี้แปลว่า *ระดับ* ของค่า $p(e)$ ที่ทำนายได้เลื่อนไปตามเวลา (ทำนายแม่นน้อยลงถ้าดู ค่าสัมบูรณ์) แต่ *อันดับ* ระหว่าง CVE ยังรักษาไว้ได้ (ตรวจสอบจริงด้วย `scripts/audit_data.py`) ข้อสรุปเชิงปฏิบัติคือ ไม่ควรอ้างว่าฟังก์ชันนี้ทำนายค่า EPSS ของ CVE ใหม่ได้แม่น ใช้ได้เพียงเพื่อจัดอันดับเท่านั้น

4.1.3 ความคลาดเคลื่อนในหน่วย log

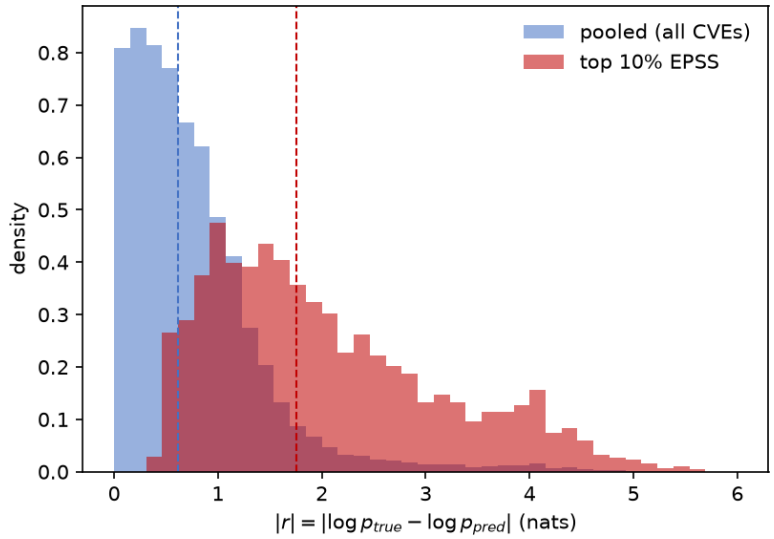
ผลข้างต้นชวนให้ถามต่อว่า ความคลาดเคลื่อนขนาดนี้กระทบ *อันดับเส้นทาง* มากแค่ไหน จริง ๆ เพราะต้นทุนของเส้นทางคือผลรวมของ $-\log p$ บน edge ตลอดเส้นทางนั้น ตัวเลขที่ บอกว่าอันดับเส้นทางจะเปลี่ยนหรือไม่จึงไม่ใช่ค่าสหสัมพันธ์ ρ อย่างที่อาจคิด แต่เป็นการ กระจายของ $r = \log p_{\text{จริง}} - \log p_{\text{ทำนาย}}$ ต่างหาก เพราะการแปลง แบบ monotone ต่อ edge คือยังรักษาลำดับของแต่ละ edge ไว้ ก็ยังสามารถสลับอันดับของ *เส้นทาง* ได้อยู่ดี ตัวอย่างเช่น เส้นทางที่มี 2 edge ซึ่งแต่ละเส้น $p = 0.9$ จะได้ต้นทุนรวม เทียบเท่า 0.81 ซึ่งชนะเส้นทางที่มี 1 edge ที่ $p = 0.8$ แต่ถ้าแปลงค่าแบบ $0.9 \mapsto 0.5$ และ $0.8 \mapsto 0.45$ (ยังคงรักษาลำดับ 0.9 มากกว่า 0.8 ไว้) อันดับของสองเส้นทางนี้กลับ สลับกันทันที

อันดับของเส้นทางจะคงเดิมพอดีก็ต่อเมื่อฟังก์ชันที่ fit เป็น power law ($\hat{p} \propto p^k$) เท่านั้น จึงนำค่ามัธยฐานของ r ออกไปก่อน (ค่านี้คือ k) แล้ววัดว่าที่เหลือกระจาย กว้างแค่ไหน ค่า r ที่วัดนี้มีหน่วยเป็น nats เพราะคำนวณจาก log ฐาน e (ลอการิทึม ธรรมชาติ) ไม่ใช่ฐาน 2 แบบ bit ที่คุ้นเคยกันทั่วไป เลือกใช้ฐาน e เพราะสะดวกต่อการแปลง กลับเป็นตัวคูณบนความน่าจะเป็น โดยตรงด้วย e^r (ตัวอย่างการแปลงกลับอยู่ในย่อหน้าถัดไป) ตารางที่ 3 รายงานค่าสัมบูรณ์ $|r|$ จากการทำนายแบบ 5-fold out-of-fold (แบ่งข้อมูลเป็น 5 ส่วนเท่า ๆ กัน ฝึกโมเดลด้วย 4 ส่วน แล้วทำนายส่วนที่เหลือที่โมเดลไม่เคย เห็น สลับไปจนครบทั้ง 5 ส่วน วิธีนี้ทำให้ได้ค่าทำนายของ CVE ทุกตัวโดยไม่มีตัวไหนถูกโมเดล “จำ” มาก่อนเลย) บน CVE ทั้ง 46,360 รายการ โดยแยกกลุ่มที่ EPSS สูงที่สุด 10% ออกมา ดูต่างหาก เพราะ edge ที่มีโอกาสสูงคือ edge ที่มักตัดสินว่าเส้นทางไหนถูกที่สุด

ตารางที่ 3 ความคลาดเคลื่อน $|r|$ ในหน่วย nats แยกตามช่วงของ EPSS จริง

กลุ่ม	n	$\|r\|$ มัธยฐาน	$\|r\|$ เปอร์เซ็นไทล์ที่ 90
ทั้งหมด (pooled)	46,360	0.587	1.483
ล่าง 90%	41,724	0.523	1.172
EPSS สูงสุด 10%	4,636	**1.938**	**3.980**

รูปที่ 3 แสดงการกระจายของ $|r|$ ทั้งสองกลุ่มซ้อนกัน เห็นได้ชัดว่าแท่งสีฟ้า (กลุ่ม pooled ทั้งหมด) กระจุกตัวอยู่ใกล้ศูนย์ ขณะที่แท่งสีแดง (กลุ่ม EPSS สูงสุด 10%) ทั้ง เตี้ยกว่าและแผ่กว้างไปทางขวามากกว่าอย่างชัดเจน เส้นประแนวตั้งคือค่ามัธยฐานของแต่ละกลุ่ม ซึ่งห่างกันเห็นได้ด้วยตาเปล่า



รูปที่ 3 การกระจายของ $|r| = |\log p_{\text{จริง}} - \log p_{\text{ทำนาย}}|$ เทียบระหว่าง กลุ่ม pooled ทั้งหมดกับกลุ่ม EPSS สูงสุด 10% เส้นประคือค่ามัธยฐานของแต่ละกลุ่ม

ตัวเลขยืนยันความกังวลข้างต้น: กลุ่มที่ EPSS สูงสุด 10% คลาดเคลื่อนมากกว่ากลุ่มล่าง 90% ประมาณ 3.7 เท่า (1.938 เทียบกับ 0.523) ถ้ารายงานแค่ค่า pooled เพียงค่าเดียว (0.587) จะ ทำให้ฟังก์ชันดูแม่นเกินจริงไปมาก เมื่อแปลงกลับเป็นตัวคูณบนความน่าจะเป็น ค่า 1.938 nats เทียบเท่า $e^{1.938} \approx 6.95$ เท่าบน $p(e)$ ขณะที่แถบการรบกวน ± 10–50% ที่ข้อเสนอโครงงานกำหนดไว้มีขนาดมากสุดเพียง $\log 1.5 \approx 0.405$ nats (เทียบเท่า 1.5 เท่า) พูดอีกแบบคือ แถบที่กำหนดไว้ในข้อเสนอแคบกว่าความคลาดเคลื่อนที่วัดได้จริงประมาณ 4.8 เท่าที่มัธยฐาน และ 9.8 เท่าที่เปอร์เซ็นไทล์ที่ 90 ข้อสรุปที่ถูกต้องจึงควรเขียนว่า “แคบกว่า 4–10 เท่า แล้วแต่เปอร์เซ็นไทล์” ไม่ใช่ “หนึ่ง order of magnitude” ลอย ๆ เพราะตัวเลขนั้น เป็นจริงเฉพาะที่

เปอร์เซ็นไทล์ที่ 90 เท่านั้น

4.1.4 ผลต่อความสามารถในการแยกอันดับเส้นทาง

คำถามสุดท้ายของหัวข้อนี้คือ ความคลาดเคลื่อนขนาดนี้ทำให้สองเส้นทางแยกอันดับกันไม่ออก เมื่อไร หลักการคือ ถ้า $|r| \leq \delta$ บนทุก edge แล้ว เส้นทางสองเส้นที่มีความยาว $|P|$ และ $|Q|$ จะยังรักษาอันดับจริงไว้ได้ ก็ต่อเมื่อผลต่างต้นทุนที่แท้จริงของทั้งสองมากกว่า $\delta\ (|P| + |Q|)$ ตารางที่ 4 แสดงขั้นต่ำของผลต่างนั้น โดยใช้ δ เป็น ทั้งค่ามัธยฐานและเปอร์เซ็นไทล์ที่ 90 ที่วัดได้จากหัวข้อก่อน

ตารางที่ 4 ผลต่างต้นทุนขั้นต่ำ (nats) ที่ทำให้สองเส้นทางยาวเท่ากันยังรักษาอันดับไว้ได้

	ค่า pooled		EPSS สูงสุด 10%	
ความยาวเส้นทาง	มัธยฐาน	P90	มัธยฐาน	P90
2 edge	2.35	5.93	7.75	15.92
4 edge	4.70	11.86	15.51	31.84
8 edge	9.39	23.72	31.01	63.68

ยกตัวอย่างเส้นทางยาว 4 edge: ผลต่างต้นทุน 4.70 nats เทียบเท่าความน่าจะเป็นจริงที่ต่างกัน $e^{4.70} \approx 109$ เท่า (คิดจาก edge คุณภาพระดับมัธยฐาน) และถ้า edge ที่ตัดสินอยู่ใน กลุ่มหาง (EPSS สูงสุด) ต้องต่างกันถึงประมาณ 5.4 ล้านเท่า จึงจะยังแยกอันดับได้ สรุปได้ว่า น้ำหนักจาก fit แยกอันดับเส้นทางได้เฉพาะคู่ที่ความน่าจะเป็นจริงต่างกันมากเท่านั้น และแทบแยกไม่ได้เลยเมื่อ edge ที่ตัดสินอยู่ในกลุ่มที่ EPSS สูง อย่างไรก็ตาม ตัวเลขในหัวข้อนี้เป็นเพียงเงื่อนไข*จำเป็น*เท่านั้น ยังไม่ได้วัดผลกระทบจริงที่เกิดขึ้น ผลกระทบจริงต่ออันดับจุดตัดถูก วัดโดยตรงในหัวข้อ 4.6

4.2 การเปรียบเทียบ Dominator Tree กับ Weighted Min-Cut (วัตถุประสงค์ข้อ 3)

วัตถุประสงค์ข้อ 3 ถามว่าจุดตัดสองนิยามที่ออกแบบไว้ในหัวข้อ 2.2.5 และ 2.2.6 ให้คำตอบตรงกันหรือไม่ นิยามแรกคือ post-dominator เป็น วิธีเชิงโครงสร้างล้วน ไม่อ่านน้ำหนักเลย คืนเซตของ node ที่อยู่บน*ทุก*เส้นทาง นิยามที่สอง คือ weighted node min-cut คืนชุด node ที่ตัดแล้วผู้โจมตีไปไม่ถึงเป้าหมาย โดยมีต้นทุนรวม ต่ำที่สุด หัวข้อนี้เทียบสองนิยามนี้บนกราฟชุดเดียวกันทั้ง 864 กราฟถ่วงน้ำหนัก

4.2.1 สามนิยามของจุดตัด

ก่อนจะเทียบกับ min-cut ได้ ต้องตกลงก่อนว่า “จุดตัด” หมายถึงนิยามใดในสามนิยามที่กล่าว ไว้ในหัวข้อ 2.2.5 เพราะทั้งสามให้เซตที่ขนาดต่างกันมาก

ตารางที่ 5 สามนิยามของจุดตัดบน 864 กราฟถ่วงน้ำหนัก

นิยาม	กราฟที่เซตไม่ว่าง	ขนาดเซตเฉลี่ย
global (ทุก entry ถึงทุก target)	8.3%	4.8
per-target (ทุก entry ถึง target เดียว)	100%	5.8
per-pair (หนึ่งคู่ entry–target)	100%	9.2

นิยาม global เข้มที่สุด เพราะจุดตัดต้องอยู่บนทุกเส้นทางของทุกคู่พร้อมกัน จึงพบเพียง 18 จาก 216 โครงสร้าง (8.3%) เท่านั้น ส่วน per-target และ per-pair ไม่ว่างถึง 100% แต่ตัวเลข นี้ต้องอ่านด้วยความระมัดระวัง เพราะเป็น artifact ของตัวสร้างกราฟ ไม่ใช่ผลที่สะท้อนเครือข่าย จริง: ในกราฟที่สร้างขึ้น target แต่ละตัวมี in-edge เพียงเส้นเดียวโดยโครงสร้าง (ดูหัวข้อ 2.2.4) node ต้นทางของ edge นั้นจึงครอบทุกเส้นทางที่เข้าถึง target เสมอ ค่า 100% จึงสะท้อนวิธีสร้างกราฟ ไม่ใช่คุณสมบัติของเครือข่ายจริง ด้วยเหตุนี้รายงานฉบับนี้จึง ใช้นิยาม global เป็นพาดหัวหลัก และกำกับข้อจำกัดนี้ไว้ทุกครั้งที่ต้องอ้างถึง per-target หรือ per-pair

4.2.2 จุดตัดเชิงโครงสร้างเป็นกรณีสุดขั้วของชั้นความน่าจะเป็น

ก่อนเทียบตัวเลขจริง มีความสัมพันธ์หนึ่งที่ควรรู้ไว้ก่อน เพราะช่วยอธิบายว่าทำไมสองวิธีจึงควร ใกล้เคียงกัน: post-dominator set พิสูจน์ได้ว่าเท่ากับเซต $\{v : \Delta(v) = \infty\}$ พอดี (คือเซตของ node ที่ถ้าลบแล้วผู้โจมตีไปไม่ถึงเป้าหมายเลย ต้นทุนจึงเพิ่มเป็นอนันต์) นั่นแปลว่า จุดตัดเชิงโครงสร้างไม่ใช่วิธีที่แยกขาดจากชั้นความน่าจะเป็นเลย แต่เป็นกรณีสุดขั้วของมันเอง คุณสมบัตินี้ตรวจสอบแล้วด้วย brute force บนกราฟสุ่ม 300 กราฟ (`tests/test_bruteforce.py`) และพบว่าจำนวน node ที่ $\Delta(v) = \infty$ เท่ากับ ขนาดของ global set ทุกกราฟจริง ๆ

4.2.3 ความตรงกันของ dominator กับ min-cut

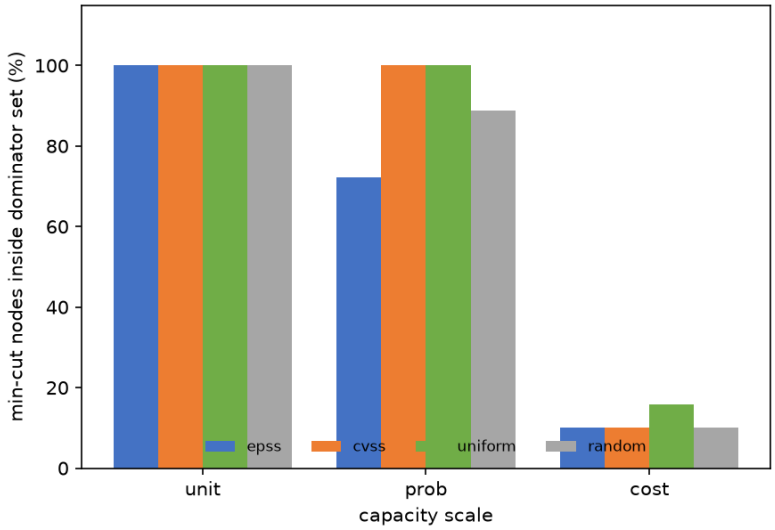
การเทียบจริงทำได้เฉพาะบนกราฟ 18 โครงสร้างที่ global set ไม่ว่างเท่านั้น (เซตว่างเทียบกัน ไม่ได้ เพราะค่า `disagreement` บนเซตว่างจะเป็น 1 โดยปริยายเสมอ ซึ่งไม่ได้บอกอะไร เกี่ยวกับความตรงกันของสองวิธีจริง ๆ) ฝั่ง min-cut รายงานสาม capacity ตามการตัดสินใจ ในหัวข้อ 2.2.6

ตารางที่ 6 แสดงสัดส่วนที่ node ใน min-cut เป็นสมาชิกของ dominator set ด้วยกันด้วย

ตารางที่ 6 สัดส่วน node ใน min-cut ที่อยู่ใน dominator set (18 โครงสร้างที่ global ไม่ว่าง)

capacity	ขนาด cut เฉลี่ย	cut อยู่ใน dominator set	บทบาท
unit	1.0	100%	หน่วยตรงกับ dominator
prob	1.1	72% EPSS, 100% อื่น	ค่าตั้งต้น
cost	3.0	10%	degenerate

รูปที่ 4 แยกตัวเลขเดียวกันตามโมเดลน้ำหนักให้เห็นชัดเจนขึ้น: แท่งของ capacity `unit` สูงเท่ากันทั้งสี่สี (100% ทุกโมเดล ตามที่คาด) แท่งของ `prob` มีเฉพาะแท่งสีน้ำเงิน (epss) ที่เตี้ยกว่าเพื่อนอย่างเห็นได้ชัด ส่วนแท่งของ `cost` เตี้ย ลงมาทุกสีพร้อมกัน ซึ่งคือสัญญาณของการเสื่อม (degenerate) ที่อธิบายในข้อสาม



รูปที่ 4 สัดส่วน node ใน weighted min-cut ที่อยู่ใน dominator set แยกตามสเกล capacity และโมเดลน้ำหนัก (18 โครงสร้างที่ global set ไม่ว่าง)

ตัวเลขในตารางนี้อ่านได้เป็นสามประเด็น

1. **capacity แบบ `unit` ตรงกับ dominator set 100%** ผลนี้เป็นไปตามที่ คาดไว้ เพราะ `unit` นับแค่จำนวน node ที่ต้องตัด ไม่สนน้ำหนักเลย ซึ่งวัดในหน่วย เดียวกับ dominator ที่ก็ไม่อ่านน้ำหนักเหมือนกัน
2. **capacity แบบ `prob` ตรงกับ dominator 100% เมื่อใช้น้ำหนัก CVSS หรือ uniform**

แต่ลดเหลือ 72% เมื่อใช้น้ำหนัก EPSS กล่าวคือใน 28% ของกรณีที่ใช้น้ำหนัก EPSS min-cut เลือก node ที่ไม่ใช่ dominator เพราะ node นั้นเป็นคอขวดของ*ความน่าจะเป็น* แม้จะไม่ได้อยู่บนทุกเส้นทางก็ตาม ข้อค้นพบหลักของวัตถุประสงค์ข้อ 3 อยู่ตรงนี้เอง: น้ำหนักเชิงประจักษ์จริงให้คำตอบต่างจากวิธีเชิงโครงสร้างล้วน ๆ ได้จริง ไม่ใช่แค่ในทางทฤษฎี

3. **capacity แบบ `cost` เสื่อม (degenerate)** เพราะ edge เชิงโครงสร้างมี $p = 1$ จึง $w = 0$ เสมอ ทำให้ node ที่เข้าถึงได้ฟรีถูกตัดด้วยต้นทุนศูนย์ min-cut จึงยุบไปเลือก เซตต้นทุนศูนย์ที่เลือกได้โดยพลการ (ยืนยันด้วยสัดส่วน cut ที่มีต้นทุนรวมเป็นศูนย์พอดีถึง 77% ของแถว) ด้วยเหตุนี้ `cost` จึงรายงานไว้เป็นเพียงคอลัมน์ sensitivity เท่านั้น ไม่ใช่ค่าหลักของการเปรียบเทียบ

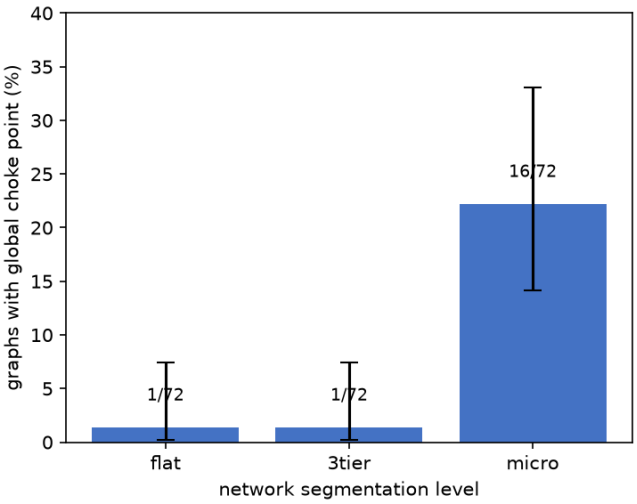
4.2.4 ระดับการแบ่งส่วนเครือข่ายกับการเกิดจุดตัด

จากผลข้างต้นที่ว่า global choke point พบได้เพียง 8.3% ของกราฟทั้งหมด คำถามต่อมาคือ กราฟแบบไหนเป็นส่วนใหญ่ของ 8.3% นั้น ข้อเสนอโครงงานถามคำถามนี้ในรูปแบบว่าเครือข่าย ต้องแบ่งส่วนละเอียดแค่ไหน จุดตัดที่การันตีได้จึงจะปรากฏ คำถามนี้ตอบได้จากสัดส่วนที่ global set ไม่ว่าง แยกตามระดับการแบ่งส่วน รายงานเป็นจำนวนกราฟพร้อมช่วงความเชื่อมั่น 95% แบบ Wilson แทนที่จะเป็นเปอร์เซ็นต์ลอย ๆ เพราะจำนวนกราฟที่เข้าเงื่อนไขในแต่ละกลุ่ม มีไม่มาก

ตารางที่ 7 สัดส่วนกราฟที่มี global choke point แยกตามการแบ่งส่วน (72 กราฟต่อระดับ)

การแบ่งส่วน	กราฟที่มีจุดตัด	ช่วงความเชื่อมั่น 95% (Wilson)
flat (ไม่แบ่ง)	1 / 72	0.2–7.5%
3-tier	1 / 72	0.2–7.5%
micro-segmentation	16 / 72	14–33%
รวม	18 / 216	5.3–12.8%

รูปที่ 5 วาดตัวเลขเดียวกันเป็นแท่งพร้อมเส้นแสดงช่วงความเชื่อมั่น: เห็นได้ชัดว่า แท่งของ micro-segmentation สูงกว่าอีกสองแท่งมาก และเส้นช่วงความเชื่อมั่น (เส้นตั้งสีดำ) ของ มันไม่แตะกับอีกสองแท่งเลย ซึ่งคือความหมายของ “แตกต่างกันอย่างมีนัยสำคัญ” ที่กล่าวถึง



รูปที่ 5 สัดส่วนกราฟที่มี global choke point แยกตามการแบ่งส่วนเครือข่าย พร้อมช่วง ความเชื่อมั่น 95% แบบ Wilson ตัวเลขกำกับคือจำนวนกราฟที่มีจุดตัด/กราฟทั้งหมดในกลุ่ม

ตารางที่ 7 ให้คำตอบที่ชัดเจน: micro-segmentation ให้จุดตัดที่การันตีได้บ่อย กว่าอีกสองแบบอย่างมีนัยสำคัญ (ช่วงความเชื่อมั่นของมันไม่ซ้อนทับกับอีกสองแบบเลย) ซึ่งตรง กับสัญชาตญาณ: เครือข่ายที่แบ่งส่วนละเอียดบีบให้ทุกเส้นทางต้องลอดผ่าน node เชื่อมเพียงไม่ กี่ตัว node เหล่านั้นจึงกลายเป็นคอขวดที่ทุกเส้นทางต้องผ่านจริง ๆ ส่วน flat และ 3-tier มี เส้นทางสำรองมากเกินกว่าจะมีจุดตัดร่วมที่ทุกคู่ต้องผ่านเหมือนกันหมด

อย่างไรก็ตาม ถึงแม้ micro จะให้ผลดีที่สุด ก็ยังมีจุดตัดเพียง 22% ของกราฟในกลุ่มนั้นเท่านั้น การพึ่งจุดตัดที่การันตีได้จึงใช้ได้จริงเฉพาะส่วนน้อยของกราฟ ส่วนที่เหลือต้องใช้การจัดอันดับ $\Delta(v)$ (หัวข้อ 2.2.5) แทนการหาเซตที่การันตีได้แบบเข้มงวด

มีข้อสังเกตด้านความแปรปรวนที่ควรบันทึกไว้ด้วย: grid รุ่นก่อนของโครงงานนี้ (135 โครงสร้าง ใช้ seed คนละชุด) เคยให้ค่า micro-segmentation สูงถึง 44% ตัวเลขที่ต่างกันมากขนาดนี้ ระหว่างสองรันย้ำว่าสัดส่วนประเภทนี้มีความแปรปรวนสูง เมื่อจำนวนกราฟที่เข้าเงื่อนไขมีน้อย จึงต้องรายงานเป็นจำนวนกราฟพร้อมช่วงความเชื่อมั่นเสมอ ไม่ใช่ตัวเลขจุดเดียวแบบที่เคยทำ

4.3 ความเสถียรของอันดับเมื่อน้ำหนักถูกรบกวน (วัตถุประสงค์ข้อ 4)

ผ่านวัตถุประสงค์ข้อ 2 และ 3 มาแล้ว คำถามของวัตถุประสงค์ข้อ 4 คือ ถ้าน้ำหนักที่คำนวณได้ ในหัวข้อ 4.1 มีความคลาดเคลื่อนจริง อันดับจุดตัดที่ได้จะยังเชื่อถือได้อยู่ไหม การทดลองนี้รบกวนน้ำหนักด้วยสัญญาณรบกวนสองรูปแบบ แบบแรกคือสมมาตร ± 10– 50% ตามที่ข้อเสนอโครงงานระบุไว้ตรง ๆ แบบที่สองคือเชิงประจักษ์ (empirical) ซึ่งสุ่มจาก residual จริงของการ fit

ที่วัดได้ในหัวข้อ 4.6 คูณด้วยสเกล 0.5 ถึง 8 เท่า (สเกล ตั้งแต่ 4 เท่าขึ้นไปถูกเพิ่มเข้ามาเพื่อหาว่าอันดับจะเริ่มพังที่ระดับใด ซึ่งเกินกว่าที่ข้อเสนอเดิม กำหนดไว้) ทั้งสองแบบวัดด้วย Jaccard@5 เทียบอันดับก่อนและหลังรบกวน บนกราฟเดิมและ โมเดลน้ำหนักเดิม

ตารางที่ 8 Jaccard@5 เฉลี่ยเมื่อรบกวนน้ำหนัก (ทุกโมเดลน้ำหนักรวมกัน)

	แบบสมมาตร (±)				
	10%	20%	30%	50%	
Jaccard@5	0.979	0.968	0.964	0.954	
	แบบเชิงประจักษ์ (เท่าของ residual จริง)				
	0.5×	1×	2×	4×	8×
Jaccard@5	0.968	0.952	0.935	0.920	0.912

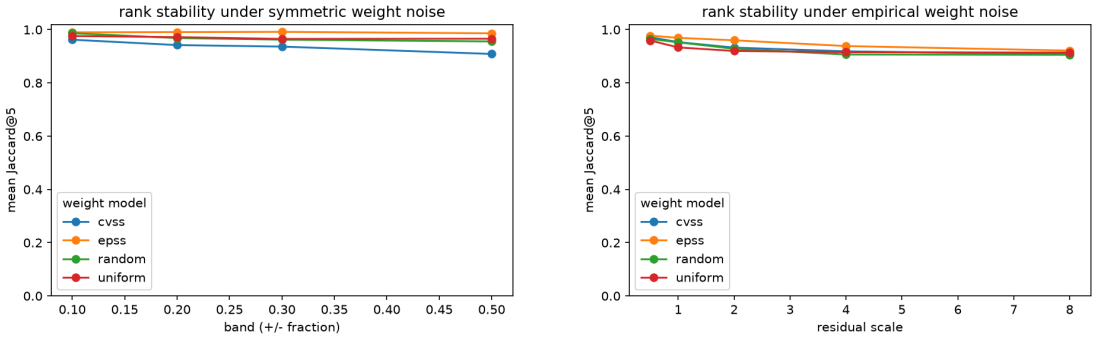
ผลที่ได้ชัดเจนและค่อนข้างน่าแปลกใจ: ไม่พบจุดที่อันดับพังเลยแม้ขยายสัญญาณรบกวนไปถึง 8 เท่าของ residual จริง Jaccard@5 ลดลงอย่างช้า ๆ จาก 0.979 ที่ $\pm 10\%$ เหลือ 0.954 ที่ $\pm 50\%$ และเหลือเพียง 0.912 ที่ความคลาดเคลื่อนใหญ่กว่าที่วัดได้จริงถึง 8 เท่า ผลนี้ สอดคล้องกับที่พบในหัวข้อ 4.6 ว่าอันดับของ node ทนทานกว่าที่ขนาดของ fit error เพียงอย่างเดียวจะทำนายได้

เพื่อตรวจว่าความทนทานนี้เป็นคุณสมบัติของโมเดลน้ำหนักใดโมเดลหนึ่งโดยเฉพาะ หรือเป็นคุณสมบัติของโครงสร้างกราฟเอง ตารางที่ 9 จึงแยกผลตามโมเดลน้ำหนัก

ตารางที่ 9 Jaccard@5 แยกตามโมเดลน้ำหนัก (แบบสมมาตร $\pm 50\%$ และเชิงประจักษ์ $8\times$)

	epss	cvss	uniform	random (null)
สมมาตร $\pm 50\%$	0.986	0.909	0.966	0.956
เชิงประจักษ์ $8\times$	0.921	0.908	0.914	0.905

รูปที่ 6 วาดเส้นแนวโน้มของทั้งสี่โมเดลน้ำหนักซ้อนกัน ทั้งแบบรบกวน สมมาตรและแบบเชิงประจักษ์ เส้นทั้งสี่เส้นวิ่งแนบชิดกันตลอดทั้งกราฟ แทบไม่มีเส้นไหนแยกตัว ออกไปอย่างเห็นได้ชัด ภาพนี้ยืนยันข้อสรุปที่ว่าความทนทานไม่ได้ขึ้นกับว่า น้ำหนักนั้นมาจากข้อมูลจริงหรือสุ่มขึ้นมาเฉย ๆ



รูปที่ 6 Jaccard@5 เฉลี่ยเมื่อรบกวนน้ำหนักแบบสมมาตร (ซ้าย) และแบบเชิงประจักษ์ (ขวา) แยกตามโมเดลน้ำหนักทั้งสี่แบบ

คำตอบคือคุณสมบัติของโครงสร้างกราฟเอง ไม่ใช่ของโมเดลน้ำหนัก แม้แต่โมเดล `random` ที่สุ่มน้ำหนักทิ้งทั้งหมดโดยไม่สนข้อมูลจริงเลย ก็ยังให้ Jaccard@5 สูงถึง 0.905–0.956 เมื่อถูกรบกวนเพิ่ม พอ ๆ กับโมเดลที่มีข้อมูลจริงรองรับ นั่นแปลว่าความทนทานที่ สังเกตได้ไม่ได้มาจากคุณภาพของน้ำหนักที่ใช้เลย แต่มาจากโครงสร้างของกราฟเอง: มีจำนวน node จำกัดที่อยู่บนเส้นทางถูกสุดได้ และการรบกวนน้ำหนักเพียงเล็กน้อยไม่พอจะเปลี่ยนว่า node ใดอยู่ในกลุ่มนั้น ข้อสรุปนี้ต้องอ่านคู่กับหัวข้อถัดไปด้วย เพราะหัวข้อถัดไปจะแสดงว่า อันดับไม่ได้ทนทานต่อทุกอย่าง มันไม่ทนทานต่อ*โครงสร้าง*ที่เปลี่ยนไปเลย

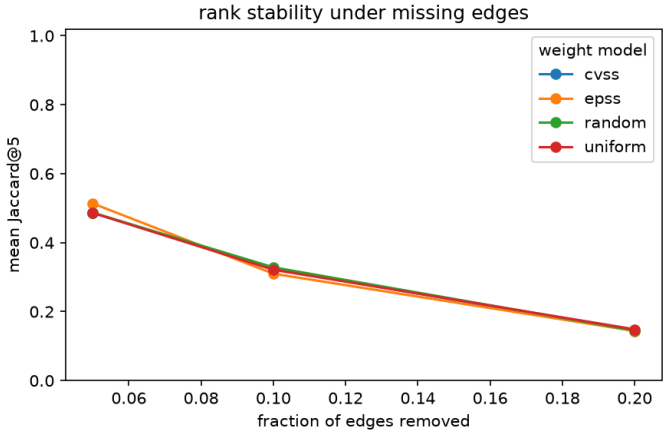
4.4 ความเสถียรของอันดับเมื่อ edge ขาดหาย

ถ้าอันดับทนทานต่อ *น้ำหนัก* ที่ผิด แล้วมันทนทานต่อ *ข้อมูลที่ขาดหาย* ด้วยหรือไม่ เพราะในโลกจริงการเก็บข้อมูลเครือข่ายมักไม่ครบ 100% การทดลองนี้จำลองสถานการณ์นั้น ด้วยการลบ edge แบบสุ่ม 5, 10 และ 20% ของกราฟ ข้อแตกต่างสำคัญจากการรบกวนน้ำหนัก ในหัวข้อก่อนคือ การลบ edge เปลี่ยน*โครงสร้าง*ของกราฟโดยตรง และอาจตัดบางคู่ (entry, target) ออกจากกันไปเลยทั้งคู่ ไม่ใช่แค่เปลี่ยนต้นทุนของเส้นทางที่มีอยู่แล้ว

ตารางที่ 10 ผลของการลบ edge แบบสุ่ม

edge ที่ลบ	Jaccard@5 ดิบ	Kendall τ	คู่ที่ยังถึงกัน	Jaccard@5 เงื่อนไข (n กราฟ)
5%	0.493	0.672	69%	0.749 (177)
10%	0.321	0.526	47%	0.650 (97)
20%	0.145	0.371	20%	0.460 (17)

รูปที่ 7 วาดเส้น Jaccard@5 ดิบเทียบกับสัดส่วน edge ที่ลบออก เส้นนี้ ลาดลงเร็วกว่าเส้นรบกวนน้ำหนักในรูปที่ 6 มาก ทั้งที่แกนนอนของทั้งสอง รูปมีสเกลเดียวกันคือ “สัดส่วนที่เปลี่ยนแปลง” การเทียบความชันของสองเส้นนี้ด้วยตาคือหลักฐาน ที่สนับสนุนข้อสรุปว่า edge ที่ขาดหายกระทบรุนแรงกว่าน้ำหนักที่คลาดเคลื่อน



รูปที่ 7 Jaccard@5 เฉลี่ย (ดิบ) เมื่อลบ edge แบบสุ่ม 5, 10 และ 20% แยกตามโมเดล น้ำหนักทั้งสี่แบบ

คอลัมน์ “ดิบ” คือ Jaccard@5 เฉลี่ยของทุกกราฟ แต่ตัวเลขนี้ปนผลสองอย่างเข้าด้วยกันโดยไม่ ได้ตั้งใจ: ส่วนหนึ่งคืออันดับเปลี่ยนเพราะต้นทุนเส้นทางเปลี่ยนจริง ๆ อีกส่วนหนึ่งคือคู่ (entry, target) บางคู่หลุดจากกันไปเลย ทำให้ node ที่เคยติดอันดับหายไปโดยสิ้นเชิงแบบที่ไม่เกี่ยวกับคุณภาพน้ำหนักเลย เพื่อแยกสองผลนี้ออกจากกัน คอลัมน์ “เงื่อนไข” จึงคำนวณเฉพาะกราฟที่ยัง คงคู่ (entry, target) ไว้ได้อย่างน้อย 90% หลังลบ edge แล้ว แม้การกรองแบบนี้จะทำให้ Jaccard@5 สูงขึ้นมาก (0.749 เทียบกับ 0.493 ที่ลบ 5%) แต่ค่าก็ยังลดลงต่อเนื่องเมื่อลบ edge มากขึ้นอยู่ดี และจำนวนกราฟที่ผ่านเงื่อนไขนี้ลดฮวบเหลือเพียง 17 กราฟที่ระดับลบ 20% ทำให้ ตัวเลขที่ระดับนั้นมีความแปรปรวนสูงมาก (ช่วงความเชื่อมั่น 95% กว้างถึง 0.32–0.66)

เมื่อเทียบตัวเลขจากทั้งสองหัวข้อนี้เข้าด้วยกัน จะเห็นข้อสรุปที่สำคัญ: edge ที่ขาดหายกระทบ อันดับรุนแรงกว่าน้ำหนักที่คลาดเคลื่อนมาก ลบ edge เพียง 5% ก็ลด Jaccard@5 ลงมากกว่า การรบกวนน้ำหนักถึง 8 เท่าของ residual จริงเสียอีก (0.493 เทียบกับ 0.912 ในหัวข้อก่อน) ข้อสรุปเชิงปฏิบัติคือ ความสมบูรณ์ของข้อมูล (การเก็บ edge ให้ครบ) สำคัญกว่าความแม่นยำของน้ำหนักบน edge ที่เก็บมาได้แล้วเสียอีก

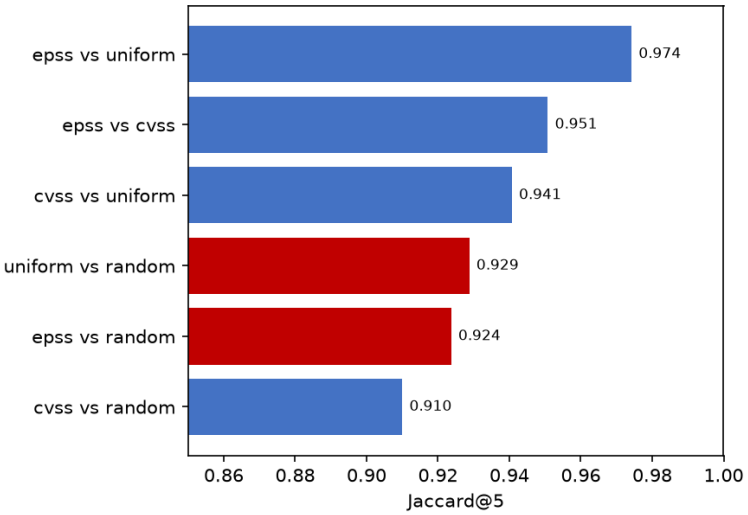
4.5 การเปรียบเทียบโมเดลน้ำหนัก 4 แบบ

ส่วนสุดท้ายของวัตถุประสงค์ข้อ 4 คือคำถามที่ข้อเสนอโครงงานถามไว้ตรง ๆ ว่า การใช้ EPSS ช่วยให้อันดับจุดตัดดีกว่าการใช้ CVSS เฉย ๆ หรือไม่ หัวข้อนี้เทียบอันดับจุดตัดที่ได้จากน้ำหนัก 4 แบบ บนกราฟโครงสร้างเดียวกัน (paired comparison): EPSS, CVSS (ใช้คะแนนฐานหารสิบ เป็นความน่าจะเป็น), uniform (ให้ $p = 0.5$ ทุก edge ที่ไม่ใช่ deterministic) และ random (สุ่ม p ใหม่ทุกครั้งโดยไม่สนข้อมูลเลย) โมเดลหลังสุดนี้ทำหน้าที่เป็น null model: อันดับส่วน ใดก็ตามที่ยังรอดอยู่แม้ใช้น้ำหนักแบบสุ่มล้วน ๆ ย่อมมาจากโครงสร้างกราฟ ไม่ใช่จากสัญญาณ ที่แฝงอยู่ในน้ำหนัก

ตารางที่ 11 Jaccard@5 ระหว่างคู่โมเดลน้ำหนัก (216 กราฟ จับคู่กัน)

คู่โมเดล	Jaccard@5	Kendall τ
epss – uniform	0.974	0.947
epss – cvss	0.951	0.884
epss – random (null)	0.924	0.834
uniform – random (null)	0.929	0.832
cvss – uniform	0.941	0.878
cvss – random (null)	0.910	0.817

รูปที่ 8 เรียงคู่โมเดลทั้งหกคู่จากน้อยไปมากตามค่า Jaccard@5 แท่งสีแดง คือคู่ที่มี `random` (null model ที่ไม่มีสัญญาณอะไรเลย) อยู่ด้วย คู่ `epss vs random` (สีแดง) กับคู่ `epss vs cvss` (สีน้ำเงิน) อยู่ใกล้กันมากบนแกน แทบไม่มีช่อง ว่างระหว่างกลุ่มสีแดงกับกลุ่มสีน้ำเงินเลย ภาพนี้สรุปได้ตรงกับข้อความถัดไปพอดี



รูปที่ 8 Jaccard@5 ระหว่างคู่โมเดลน้ำหนักทั้งหกคู่ (216 กราฟ จับคู่กัน) แท่งสีแดงคือคู่ที่มี `random` (null model) อยู่ด้วย

แถวที่ต้องอ่านคู่กันคือแถวที่สองและสาม: EPSS เทียบกับ CVSS ให้ Jaccard@5 เท่ากับ 0.951 ขณะที่ EPSS เทียบกับ *random* (ซึ่งไม่มีสัญญาณอะไรเลย) ให้ 0.924 ต่างกันเพียง 0.027 เท่านั้น ส่วนต่างที่เล็กขนาดนี้ยังไม่พอเป็นหลักฐานว่า EPSS ให้อันดับที่ “ถูกต้องกว่า” CVSS เพราะกราฟจำลองในโครงงานนี้ไม่มีค่าจริง (ground truth) ให้เทียบตั้งแต่ต้น ตัวเลขใน ตารางนี้จึงวัดได้แค่*ความต่างกันของอันดับ*ระหว่างสองน้ำหนักเท่านั้น ไม่ใช่ความถูกต้อง การที่ EPSS ใกล้เคียงกับ CVSS มากกว่าใกล้เคียงกับ random อยู่เล็กน้อย พอจะบ่งชี้ได้ว่าทั้ง สองโมเดลที่มีข้อมูลจริงรองรับให้อันดับที่คล้ายกันในระดับหนึ่ง แต่ความคล้ายนั้นเมื่อเทียบกับ null model แล้วมีขนาดเล็กมาก สรุปตามหลักการที่วางไว้ตั้งแต่ต้นโครงงานว่า การทดลองนี้วัดได้เพียง disagreement rate และความเสถียรของอันดับเท่านั้น ไม่ใช่ความ แม่นยำ

4.6 ผลของความคลาดเคลื่อนจากการ fit ต่ออันดับจุดตัด

หัวข้อนี้วัดผลจริงของสิ่งที่หัวข้อ 4.1.4 ทำนายไว้ว่าเป็นเพียงเงื่อนไขจำเป็น เท่านั้น ด้วยการทดลองตรง ๆ คือ ถ่วงน้ำหนัก CVE แต่ละตัวด้วยค่า EPSS จริง แล้วเทียบกับ ถ่วงน้ำหนักด้วยค่าที่ fit ทำนายจากเวกเตอร์ CVSS ของ CVE ตัวเดียวกัน (กราฟเดียวกันทุก ประการ โครงสร้างเดียวกัน ต่างกันแค่แหล่งที่มาของ p บน edge ที่มี CVE เท่านั้น) แล้วดูว่า อันดับจุดตัดเปลี่ยนไปมากน้อยเพียงใด ทดลองบนกราฟขนาด $n = 200$ จำนวน 45 กราฟ (รัน แยกจากกริดหลักด้วยสคริปต์ `experiments/exp6_fit_error.py` ตามขั้นตอนที่ระบุไว้ ในภาคผนวก 5.3 เหตุผลที่แยกสคริปต์ต่างหากคือการทดลองนี้ต้องรู้ค่า EPSS จริงของ CVE แต่ละตัวคู่กับค่าที่ fit ได้ ซึ่งกริด

หลักไม่ได้เก็บคู่นี้ไว้)

ตารางที่ 12 ผลของการใช้ fitted p แทน EPSS จริง บน CVE edge (n=45 กราฟ, $n=200$)

ตัวชี้วัด	ค่า
Jaccard@5	0.993
Jaccard@10	0.980
Kendall τ	0.955
เส้นทางถูกสุดเหมือนเดิม	36 / 45 (80%)
อัตราส่วนต้นทุนเส้นทางถูกสุด (มัธยฐาน, p10–p90)	1.00 (0.93–1.11)

ผลที่ได้แบ่งเป็นสองส่วนที่ดูขัดกันในตอนแรก แต่จริง ๆ แล้วสอดคล้องกันดี ส่วนแรกคือ อันดับ node (วัดด้วย Jaccard@5, Jaccard@10 และ τ) แทบไม่ขยับเลย แม้ residual ต่อ edge จะมีขนาดเฉลี่ยราว 1 nat ก็ตาม ซึ่งสอดคล้องกับผลในหัวข้อ 4.3 ที่ว่า โครงสร้างกราฟเป็นตัวกำหนดอันดับหลัก ไม่ใช่ความแม่นของน้ำหนัก ส่วนที่สองคือเส้นทางที่ ถูกสุดเพียงเส้นเดียว (ไม่ใช่อันดับ node โดยรวม) กลับเปลี่ยนไปถึง 9 จาก 45 กราฟ (20%) ทั้งที่อัตราส่วนต้นทุนรวมยังใกล้ 1 แทบทุกกรณี (มัธยฐาน 1.00)

เหตุผลที่สองผลนี้ไม่ขัดแย้งกันคือ Jaccard@5 นับแค่ว่า node ใดติดอันดับต้น ๆ โดยไม่สนว่า เส้นทางที่แท้จริงที่สุดคือเส้นไหน ขณะที่ `top_path_same` นับเฉพาะเส้นทางเดียวที่ ถูกที่สุดเท่านั้น เมื่อสองเส้นทางมีต้นทุนใกล้กันมาก การสลับตำแหน่งกันทำได้ง่ายโดยที่อันดับ node โดยรวมไม่เปลี่ยนเลย กล่าวโดยสรุป การใช้ fit แทน EPSS จริงปลอดภัยสำหรับงานจัด อันดับจุดตัด แต่ไม่ปลอดภัยสำหรับการชี้ว่า “เส้นทางโจมตีที่เป็นไปได้มากที่สุด” คือเส้นไหน เพียงเส้นเดียว

ผลลัพธ์นี้ยืนยันทิศทางที่คาดไว้ในหัวข้อ 4.1.4 (ที่ว่าเงื่อนไขจำเป็นของการ รักษาอันดับคือการกระจายของ $|r|$ ในหน่วย log) แต่ก็แสดงด้วยว่าเงื่อนไขนั้นเข้มงวดเกินไป สำหรับ*อันดับ node* โดยเฉพาะ: แม้ $|r|$ เฉลี่ย 1 nat จะมากกว่าแถบ ± 10–50% ของข้อเสนอหลายเท่า อันดับ node ก็ยังรอดอยู่ดี เพราะอันดับ node พึ่งเพียงว่า node ใดอยู่บน เส้นทางถูกสุด*ของคู่ใดคู่หนึ่ง*เท่านั้น ไม่ได้ขึ้นกับว่าเส้นทางถูกสุดเพียงเส้นเดียวคือเส้นใด ข้อควรระวังคือการทดลองนี้ครอบคลุมเฉพาะกราฟขนาด $n = 200$ เท่านั้น ไม่ใช่ทั้ง grid

4.7 สัดส่วนเส้นทางที่ข้ามโดเมน

กลับมาที่คำถามของข้อเสนอโครงงานโดยตรง: เส้นทางโจมตีที่อันตรายที่สุดวิ่งผ่านขอบเขต on-prem–cloud ในสัดส่วนเท่าใด หัวข้อนี้วัดสองค่าต่อคู่ (entry, target) ที่ไปถึงกันได้ ค่าแรก คือ

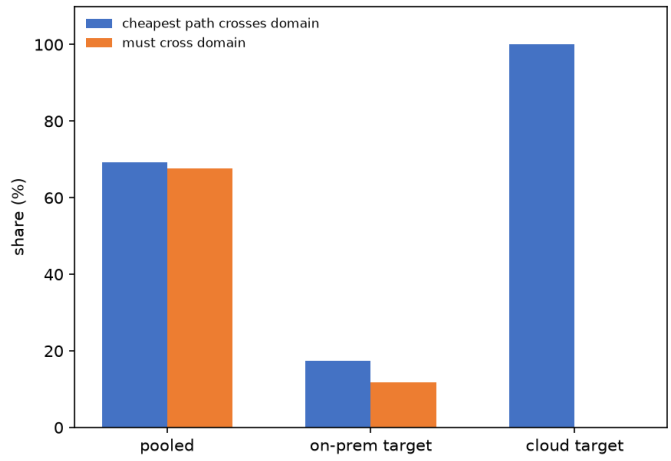
`cheapest_cross_share` คือสัดส่วนที่เส้นทางถูกสุด (ต้นทุนต่ำสุดตาม $-\log p$) ใช้ edge ข้ามโดเมนอย่างน้อยหนึ่งเส้น ค่าที่สองคือ `cross_required_share` คือสัดส่วนที่คู่นั้น*ต้อง*ข้ามโดเมนเท่านั้นจึงจะไป ถึงกันได้ ถ้าตัด edge ข้ามโดเมนทั้งหมดออก คู่นั้นจะไปถึงกันไม่ได้อีกเลย ทั้งสองค่าวัด บนน้ำหนัก EPSS บนกราฟ 216 โครงสร้าง

ก่อนอ่านตัวเลข ต้องรู้ข้อจำกัดหนึ่งก่อน: เนื่องจาก entry ทุกตัวอยู่ฝั่ง on-prem โดยการออกแบบของตัวสร้างกราฟเองตั้งแต่ต้น (หัวข้อ 2.2.4) คู่ที่มี target อยู่ฝั่ง cloud จึง*ต้อง*ข้ามโดเมนเสมอโดยโครงสร้าง (ได้ 100% ทั้งสองค่าโดยอัตโนมัติ ไม่มี ทางเป็นอื่น) ตัวเลขที่มีความหมายจริงจึงอยู่ที่คู่ซึ่ง target เป็น on-prem ด้วยกันเท่านั้น เพราะ กรณีนั้นการข้ามโดเมนเป็นทางเลือก ไม่ใช่ข้อบังคับ

ตารางที่ 13 สัดส่วนเส้นทางข้ามโดเมน แยกตาม target (น้ำหนัก EPSS, 216 กราฟ)

กลุ่ม target	เส้นทางถูกสุดข้ามโดเมน	ต้องข้ามโดเมน
รวม (pooled)	69.3%	67.6%
target ฝั่ง cloud	100%	100% (โครงสร้าง)
target ฝั่ง on-prem	**17.5%**	**11.8%**

รูปที่ 9 เทียบสามกลุ่มเป็นแท่งคู่ แท่งกลุ่ม cloud target สูงเต็ม 100% ทั้ง สองสี (เพราะเป็นผลโครงสร้างตามที่อธิบายไว้) ส่วนแท่งกลุ่ม on-prem target ต่ำกว่ามาก ทั้งที่ แท่งกลุ่ม pooled (ซึ่งปนทั้งสองกลุ่มเข้าด้วยกัน) อยู่กึ่งกลาง ความต่างระหว่างแท่งกลุ่ม pooled กับกลุ่ม on-prem target นี้เองคือเหตุผลที่ต้องแยกอ่านตามกลุ่ม target เสมอ ไม่ใช่ดูแค่ค่า pooled



รูปที่ 9 สัดส่วนเส้นทางข้ามโดเมน (น้ำหนัก EPSS) แยกตามกลุ่ม target

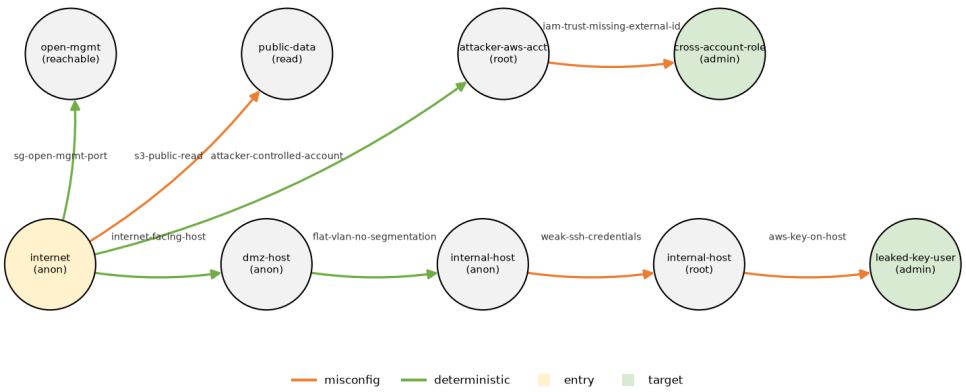
สำหรับคู่ที่ target เป็น on-prem ด้วยกัน เส้นทางถูกสุดใช้ edge ข้ามโดเมนเพียง 17.5%

ของ กรณีเท่านั้น และมีเพียง 11.8% เท่านั้นที่*ไม่มีทางเลือกอื่น*นอกจากข้ามโดเมน พูดอีกแบบ คือ ส่วนใหญ่ของเส้นทางอันตรายในเครือข่ายเดียวกันไม่จำเป็นต้องพึ่งพาการเชื่อมต่อข้าม โดเมนเลย

ตัวเลข 11.8% นี้ต้องอ่านอย่างระมัดระวัง เพราะส่วนใหญ่มาจากกราฟแบบ micro-segmentation เพียงแบบเดียว (44.7% เทียบกับ 4.8% ของ 3-tier และ 3.0% ของ flat) ที่เป็นแบบนี้เพราะ ขั้นตอนซ่อมกราฟ (repair, หัวข้อ 2.2.4) ไม่ใช่เพราะโครงสร้างธรรมชาติล้วน ๆ เมื่อ micro-segmentation ตัดทุกเส้นทางภายในออกจนบางคู่ไปไม่ถึงกัน ตัวสร้างกราฟจะเติม edge แบบ VPN/Direct Connect เข้าไปซ่อม edge แบบนี้ถูกจัดเป็น `domain="cross"` เสมอ ตัวเลข 11.8% จึงสะท้อนพฤติกรรมของตัวซ่อมกราฟบางส่วน ไม่ใช่คุณสมบัติที่พบใน เครือข่ายจริงล้วน ๆ ต้องระบุข้อจำกัดนี้กำกับไว้ทุกครั้งที่อ้างถึงตัวเลขนี้ (ดูหัวข้อ 4.10) นอกจากนี้สัดส่วนยังเพิ่มขึ้นตามความหนาแน่นของ edge ข้ามโดเมน ที่กำหนดไว้ (low 12.2%, med 18.2%, high 22.1%) ตรงตามสัญชาตญาณ: ยิ่งมี edge ข้ามโดเมนให้เลือกมาก เส้นทางถูกสุดก็ยิ่งมีโอกาสผ่าน edge เหล่านั้นมากขึ้นตามไปด้วย

4.8 ผลจาก lab จริง (วัตถุประสงค์ข้อ 5)

ทุกผลข้างต้นมาจากกราฟจำลอง คำถามสุดท้ายคือ pipeline เดียวกันนี้ใช้กับกราฟจากระบบจริงได้หรือไม่ หัวข้อนี้ยืนยันว่าใช้ได้โดยไม่ต้องแก้โค้ดวิเคราะห์แม้แต่บรรทัดเดียว กราฟที่ export จาก Neo4j/Cartography ผ่านทาง `lab/exporter/export_graph.py` มี 9 node, 8 edge, entry เดียว (`internet, anon`) และ 2 target คือ role ข้าม account บน AWS (หมายถึงกรณีที่ IAM role ของบัญชี AWS หนึ่งถูกตั้งค่าให้ role ของอีกบัญชีหนึ่งสวมสิทธิ์ แทนได้ ถ้าผู้โจมตียึด role ต้นทางได้ ก็สวมสิทธิ์ข้ามไปอีกบัญชีได้ทันทีโดยไม่ต้องเจาะอะไร เพิ่ม ตรงวัตถุประสงค์ cloud) และผู้ใช้ที่ถือ key หลุด (ตรงวัตถุประสงค์ cross-domain) รูปที่ 10 วาดกราฟนี้ทั้งใบตามที่ export ออกมาจริง เห็นได้ชัดว่าแยกเป็นสองกิ่งจาก `internet` โดยไม่มี node ร่วมกันเลยนอกจาก entry เอง นี่คือสาเหตุที่ global dominator set ของกราฟนี้เป็นเซตว่าง ตามที่อธิบายในย่อหน้าถัดไป



รูปที่ 10 กราฟจาก lab จริงทั้งใบ (9 node, 8 edge) ตามที่ `export_graph.py` ส่งออก จาก Neo4j/Cartography สีของ edge คือชนิด (`kind`) ไม่ใช่โดเมน

ตารางที่ 14 ประเภท edge ในกราฟ lab จริง (9 node, 8 edge)

ชนิด (kind, domain)	จำนวน
misconfig, cloud	2
deterministic, cloud	2
deterministic, onprem	2
misconfig, onprem	1
misconfig, cross	1

เมื่อรันกราฟนี้ผ่าน `weights.assign` และ `chokepoints` ตัวเดียวกับที่ใช้กับกราฟจำลองทุกประการ global dominator set ที่ได้คือเซตว่าง เพราะกราฟนี้แยกเป็นสองกิ่งที่ไม่มี node ร่วมกันเลยนอกจาก entry (กิ่งหนึ่งไปยัง role บน AWS โดยตรง อีกกิ่งผ่านเครือข่าย on-prem ก่อนจะข้ามไปยัง AWS) ผลลัพธ์นี้ไม่ใช่ข้อผิดพลาดของระบบแต่อย่างใด แต่เป็นข้อ ค้นพบเชิงโครงสร้างจริงตามนิยามในหัวข้อ 4.2: เมื่อพื้นผิวโจมตีมีความหลาก หลาย (คือมีสองเส้นทางที่เป็นอิสระต่อกันโดยสิ้นเชิง) จุดตัดที่การันตีทุกเส้นทางพร้อมกันจึงไม่มี อยู่จริง ต้องอ่านนิยาม per-target แทน ซึ่งให้ผลตามตารางที่ 15

ตารางที่ 15 จุดตัดต่อ target ของกราฟ lab จริง

target	จุดตัด (per-target)
role ข้าม account (AWS)	`aws:999999999999, root` (บัญชีผู้โจมตีเอง)
ผู้ใช้ที่ถือ key หลุด	ทั้งสามสถานะบนโฮสต์ on-prem: `dmz (anon)`, `internal (anon)`, `internal (root)`

ผลนี้ตรงกับที่คาดไว้: ทุกเส้นทางสู่ target ที่สอง (ผ่าน key หลุด) ต้องผ่านทั้งสามสถานะบน เครือข่าย on-prem เพราะ topology ของ lab เป็นโซ่เดียว (DMZ ⟶ internal ⟶ root ⟶ AWS) ไม่มีเส้นทางสำรองเลย ขณะที่ target แรกมีจุดตัดเดียวคือบัญชีผู้โจมตีเอง นั่นก็ เป็นจุดเริ่มต้นของกิ่งนั้นอยู่แล้วตั้งแต่ต้น

ในส่วนที่ lab นี้ยังไม่ครอบคลุม ต้องเขียนไว้อย่างตรงไปตรงมา: lab แทนที่ GNS3 ด้วยเครือข่าย Docker ธรรมดา แทนที่ Nessus ด้วย `nmap` ซึ่งให้แค่ความสามารถในการ เข้าถึงพอร์ต (reachability) ไม่มีข้อมูล CVE เลย และแทนที่ AWS จริงด้วย LocalStack เพราะ บัญชี AWS Academy Learner Lab บล็อกการสร้าง IAM resource ทั้งหมด (รายละเอียดอยู่ใน ภาคผนวก 5.3) ผลที่ตามมาคือกราฟ lab นี้ไม่มี edge ชนิด CVE แม้แต่เส้น เดียว จึงยังไม่ได้ทดสอบฝั่ง EPSS ของวัตถุประสงค์ข้อ 2 ด้วยข้อมูลจริง ทดสอบได้เฉพาะฝั่ง misconfiguration และ deterministic เท่านั้น นอกจากนี้ฝั่ง on-prem ก็ไม่ได้ผ่าน Cartography/Neo4j ด้วย (เพราะ Cartography รองรับเฉพาะ cloud) แต่อ่านข้อมูลตรงจาก `docker exec/nmap` แทน

แม้จะมีข้อจำกัดเหล่านี้ edge ข้ามโดเมนเส้นเดียวในกราฟนี้ (`aws-key-on-host`) ก็ ยังได้รับการยืนยันอย่างน่าเชื่อถือ เพราะต้องการหลักฐานจากสองแหล่งข้อมูลอิสระที่ต้องตรงกัน คือไฟล์ credential บนโฮสต์ และ node ที่ Cartography ค้นพบเองจากฝั่ง cloud ความน่าเชื่อถือของกฎนี้ทดสอบแล้วจริง: เมื่อลองแก้ไข key id บนโฮสต์ให้ไม่ตรงกับฝั่ง cloud exporter ปฏิเสธสร้าง edge นั้นทันที ซึ่งยืนยันว่า edge ข้ามโดเมนนี้เป็นข้อเท็จจริงที่ตรวจสอบได้จริง ไม่ใช่ข้อเท็จจริงที่ปลูกไว้ฝ่ายเดียว

4.9 อภิปรายผลและภัยต่อความตรงของผล

หัวข้อนี้รวบรวมปัจจัยที่อาจทำให้ตีความผลข้างต้นผิดพลาดได้ (threats to validity) เรียง ตามความสำคัญ แต่ละข้อถูกวัดหรือทดสอบจริงระหว่างการตรวจทานโครงงาน ไม่ใช่ข้อสงสัย ลอย ๆ ที่คิดขึ้นมาเฉย ๆ

1. **ตัวสร้างกราฟให้โครงสร้างแบบโซ่ (serial), ไม่ใช่เครือข่ายจริง** ในหัวข้อ 4.3 อันดับ node ทนทานต่อน้ำหนักที่ผิดมาก จนแม้แต่น้ำหนักแบบ สุ่มล้วน (`random`) ก็ยังให้อันดับใกล้เคียงกับ EPSS การที่ null model ให้ผลใกล้เคียง กับโมเดลที่มีข้อมูลจริงขนาดนี้บ่งชี้ว่าอันดับถูกกำหนดโดย*ตำแหน่งของ node ใน โครงสร้าง*เป็นหลัก ไม่ใช่คุณภาพของน้ำหนัก และนี่เป็นคุณสมบัติของกราฟที่สร้างขึ้น (ซึ่งส่วน ใหญ่มีเส้นทางสำรองน้อย) ยังไม่ได้พิสูจน์เลยว่าเครือข่ายไฮบริดจริงก็เป็นเช่นนั้นด้วย
2. **สถิติ per-target/per-pair ที่ไม่ว่าง 100% เป็น artifact ของตัวสร้างกราฟ** ตามที่กล่าวไว้ในหัวข้อ 4.2.1 เพราะ target แต่ละตัวมี in-edge เดียวโดย โครงสร้าง ตัวเลขนี้จึงไม่ควรนำไปใช้เป็นหลักฐานว่าเครือข่ายจริงมีจุดตัดต่อ target เสมอ
3. **สัดส่วน global choke point แปรปรวนสูงระหว่างรันสองครั้ง** grid รุ่นก่อน (135 โครงสร้าง ใช้ seed คนละชุด) เคยให้ค่า micro-segmentation ที่ 44% ขณะที่ grid สุดท้าย (216 โครงสร้าง) ให้ 22% ทั้งที่ใช้ตัวสร้างกราฟตัวเดียวกันเป๊ะ ความต่างนี้ย้ำว่าต้องรายงานตัวเลขกลุ่มนี้พร้อมช่วงความเชื่อมั่นเสมอ ไม่ใช่ตัวเลขจุดเดียวลอย ๆ (ตารางที่ 7)
4. **สมมติฐาน edge independence และ OR-semantics ทำให้ผล Dijkstra เป็น เพียงขอบเขตล่างของต้นทุนผู้โจมตี** (lower bound) ต้นทุนจริงไม่มีทางแพงกว่านี้ มีแต่จะเท่ากันหรือถูกกว่า ตามที่อธิบายไว้ในหัวข้อ 2.2.1 ไม่ใช่ความน่าจะเป็นที่จะ ถูกเจาะจริง ($P(\text{compromise})$) เพราะภายใต้ OR-semantics ยังมีหลายเส้นทางที่ไปถึง เป้าหมายเดียวกันได้ ความน่าจะเป็นที่แท้จริงจึงต้องมากกว่าความน่าจะเป็นของเส้นทางเดียวที่ถูกที่สุด
5. **สเกลของ p ระหว่าง edge ประเภท fitted/EPSS กับ deterministic ไม่ใช่ความหมายเดียวกัน** EPSS วัด “โอกาสถูกโจมตีที่ไหนสักแห่งในโลก” ไม่ใช่ “โอกาสสำเร็จเมื่อยืนอยู่ต้นทางแล้ว” ขณะที่ edge เชิงโครงสร้างมี $p = 1$ เสมอ การผสม สองความหมายนี้ในกราฟเดียวกันทำให้ Dijkstra เลือกเส้นทางตามน้ำหนักฝั่งที่ถูก calibrate เป็นหลัก การตัดสินใจของโครงงานนี้ (ยืนยันแล้ว) คือคงรูปแบบนี้ไว้ พร้อมรายงาน sensitivity analysis กำกับ แทนที่จะแยกสเกลออกจากกัน (รายละเอียดอยู่ในภาคผนวก 5.3)
6. **ไม่มีค่าจริง (ground truth) บนกราฟจำลอง** ดังนั้นหัวข้อ 4.5 วัดได้เพียงความต่างระหว่างอันดับของสองโมเดลน้ำหนักเท่านั้น ไม่ใช่ว่าโมเดลใด “ถูกต้อง กว่า” อีกโมเดลหนึ่ง
7. **ตัวอย่างข้อมูลสำหรับ fit ไม่ใช่การสุ่ม CVE** ตามที่กล่าวไว้ในหัวข้อ 4.1.1 แต่เป็นบล็อกต่อเนื่องจาก NVD ซึ่งปีกระจายไม่สม่ำเสมอ (ไม่มีปี 2016 เลย) และค่า R^2 ที่รายงาน (0.219) มาจากการแบ่งข้อมูลทดสอบแบบสุ่มราย CVE ซึ่งมอง โลกในแง่ดีกว่าการใช้งานจริง: เมื่อกันทั้งหน้า NVD ออกแทน R^2 ลดเหลือ 0.156 และเมื่อ ฝึกด้วยข้อมูลถึงปี 2022 แล้วทดสอบกับ

ปี 2023 เป็นต้นไป R^2 ติดลบไปเลย (-0.863) แม้ Spearman ρ จะยังอยู่ที่ 0.542 (ตรวจสอบแล้วด้วย `scripts/audit_data.py`)

4.10 ข้อจำกัด

- **ไม่มี AND-node** โมเดลนี้ใช้ OR-semantics ล้วน ตามที่ประกาศไว้ตั้งแต่ต้น การโจมตีที่ต้องอาศัยเงื่อนไขหลายอย่างพร้อมกัน (เช่น AND-node ของ MulVAL) จึงไม่ได้ถูก แสดงในกราฟเลย
- **lab ไม่มี edge ชนิด CVE** เพราะแทนที่ Nessus ด้วย `nmap` ซึ่งให้แค่ ข้อมูลการเข้าถึงพอร์ต การทดสอบฝั่ง EPSS ของวัตถุประสงค์ข้อ 2 บนระบบจริงจึงยังไม่เกิดขึ้น
- **lab ฝั่ง on-prem ไม่ผ่าน Cartography/Neo4j** แต่อ่านข้อมูลตรงจาก `docker exec`/`nmap` แทน เพราะ Cartography รองรับเฉพาะ cloud provider
- **AWS จริงใช้ไม่ได้** บัญชี AWS Academy Learner Lab บล็อกการสร้าง IAM resource ทั้งหมด (ภาคผนวก 5.3) lab จึงต้องรันบน LocalStack แทน
- **3 กลุ่มคลาส misconfiguration มีค่า p เท่ากันเป๊ะ** เพราะตกอยู่ใน CVSS cell เดียวกัน (`iam-trust-wildcard-principal`/`cicd-oidc-broad-sub`/`federated-group-overbroad-role`; `iam-passrole-excess-privilege`/`stale-service-account-cred`; `unauth-internal-service`/`weak-ssh-credentials`) จึงแยกด้วยน้ำหนักไม่ได้เลย แม้แนวคิดจะต่างกันก็ตาม
- **กราฟจำลองมีโครงสร้างแบบโซ่เป็นส่วนใหญ่** ยังไม่ได้ตรวจสอบกับ topology ของ เครือข่ายไฮบริดจริงเลยว่าใกล้เคียงกันเพียงใด

บทที่ 5 สรุป

5.1 สรุปผลตามวัตถุประสงค์

จากผลการทดลองทั้งหมดในบทที่ 4 หัวข้อนี้สรุปว่าแต่ละวัตถุประสงค์ทั้งห้าข้อของ โครงงานสำเร็จมากน้อยเพียงใด เรียงตามลำดับเดียวกับที่ประกาศไว้ในหัวข้อ 1.3

1. **โมเดลกราฟการโจมตี (สำเร็จ)** node แบบ `(resource, privilege)` และ edge สามชนิดพร้อม `domain` กำกับ (หัวข้อ 2.2.1) ใช้ได้ทั้งกราฟจำลอง 216 โครงสร้าง และกราฟจาก lab จริงโดยไม่ต้องแก้โค้ดวิเคราะห์เลย (หัวข้อ 3.3, 4.8) ความถูกต้องของ โครงสร้างถูกตรวจด้วย brute force บนกราฟสุ่ม 300 กราฟ (หัวข้อ 3.6)
2. $p(e)$ **ที่อธิบายที่มาได้ (สำเร็จบางส่วน)** ทุก edge มี `p_source` และ `p_input` กำกับจริง ฟังก์ชัน fit จาก CVSS ไปเป็น EPSS ใช้งานได้และตรวจสอบ ย้อนกลับได้ทีละ metric (หัวข้อ 4.1) แต่ความแม่นยำมีจำกัด: R^2 ในสเกล logit เท่ากับ 0.219 เท่านั้น และคลาดเคลื่อนมากที่สุดบน edge ที่มีโอกาสถูกโจมตีสูง (top-decile $|r|$ 1.938 nats เทียบ pooled 0.587) กลุ่มนี้เองที่ตัดสินอันดับเส้นทางจริง ข้อจำกัดนี้ถูก รายงานตรงไปตรงมา ไม่ปกปิด ตามที่ประกาศไว้ในหลักการของโครงงาน
3. **เทียบ dominator tree กับ weighted min-cut (สำเร็จ)** ทั้งสองวิธีคำนวณบน กราฟชุดเดียวกันด้วยหน่วยที่เทียบกันได้ (node-splitting, หัวข้อ 2.2.6) พบว่าเมื่อใช้น้ำหนัก EPSS จริง min-cut ให้คำตอบตรงกับ dominator set เพียง 72% ของกรณีที่เทียบได้ ต่างจาก เมื่อใช้น้ำหนัก CVSS หรือ uniform ที่ตรงกัน 100% (หัวข้อ 4.2.3) ข้อค้นพบหลักของวัตถุประสงค์ข้อนี้คือ วิธีเชิงโครงสร้างกับวิธีที่ใช้ข้อมูลเชิงประจักษ์จริงให้ คำตอบต่างกันได้จริง ไม่ใช่แค่ในทางทฤษฎี
4. **วัดความทนทานของอันดับ (สำเร็จ)** พบว่าน้ำหนักที่คลาดเคลื่อนไม่ทำให้อันดับ พัง แม้ขยายสัญญาณรบกวนไปถึง 8 เท่าของค่าที่วัดได้จริง (Jaccard@5 ยังอยู่ที่ 0.912, หัวข้อ 4.3) แต่ edge ที่ขาดหายกระทบรุนแรงกว่ามาก (Jaccard@5 เหลือ 0.493 เมื่อลบเพียง 5%, หัวข้อ 4.4) ส่วนคำถามว่า EPSS ช่วยได้มากกว่า CVSS หรือไม่ คำตอบคือไม่มี หลักฐานเพียงพอ เพราะกราฟจำลองไม่มีค่าจริงให้เทียบ (EPSS เทียบ CVSS ให้ Jaccard@5 0.951 ใกล้เคียงกับ EPSS เทียบน้ำหนักสุ่มล้วนที่ 0.924 มาก, หัวข้อ 4.5)
5. **lab จริงสองฝั่ง (สำเร็จบางส่วน)** pipeline เดียวกันใช้ได้กับกราฟจาก Ter-raform/LocalStack/Cartography/Neo4j จริง (หัวข้อ 4.8) แต่ไม่ครบตามที่เสนอไว้ทุก

ส่วน: ใช้ Docker network แทน GNS3, ใช้ `nmap` แทน Nessus (จึงไม่มี edge ชนิด CVE ในกราฟ lab เลย) และใช้ LocalStack แทน AWS จริง เพราะบัญชี AWS Academy Learner Lab บล็อกการสร้าง IAM resource ทั้งหมด การตัดสินใจทดแทนแต่ละจุดมีเหตุผล ที่บันทึกไว้ ไม่ใช่การข้ามแบบเงียบ ๆ (ภาคผนวก 5.3)

โดยรวม วัตถุประสงค์ข้อ 1, 3 และ 4 สำเร็จครบตามที่ตั้งไว้ ข้อ 2 และ 5 สำเร็จบางส่วน ด้วยข้อจำกัดที่ระบุไว้ชัดเจนและมีเหตุผลรองรับทุกจุด

5.2 ข้อจำกัด

ข้อจำกัดของโครงงานนี้สรุปไว้ครบถ้วนแล้วในหัวข้อ 4.10 (ท้ายบทที่ 4) ซึ่งครอบคลุมทั้ง ขอบเขตของโมเดล (ไม่มี AND-node), ข้อจำกัดของ lab (ไม่มี edge ชนิด CVE, ไม่ผ่าน Cartography ฝั่ง on-prem, ใช้ LocalStack แทน AWS จริง), ข้อจำกัดของ rubric (3 กลุ่มคลาส มีค่า p เท่ากันเป๊ะ เพราะตกอยู่ใน CVSS cell เดียวกัน) และข้อจำกัดของตัวสร้างกราฟ จำลอง (โครงสร้างแบบโซ่ ยังไม่ได้ตรวจกับ topology จริง)

5.3 แนวทางพัฒนาต่อ

- **ทดสอบกับ AWS จริง** เมื่อมีบัญชีที่ไม่ใช่ AWS Academy Learner Lab เพื่อยืนยัน ว่าผลจาก lab ไม่ได้ขึ้นกับข้อจำกัดเฉพาะของ LocalStack
- **ใช้ Nessus หรือ OpenVAS ที่ใช้งานได้จริง** เพื่อให้กราฟ lab มี edge ชนิด CVE อย่างน้อยหนึ่งเส้น และทดสอบฝั่ง EPSS ของวัตถุประสงค์ข้อ 2 บนระบบจริงได้
- **เขียนตัวเก็บข้อมูลฝั่ง on-prem ที่ผ่าน graph database** แทนการอ่านตรงจาก `docker exec/nmap` เพื่อให้ pipeline ทั้งสองฝั่งสมมาตรกัน
- **รองรับ AND-node บางส่วน** สำหรับกรณีที่พบบ่อยในโลกจริง เช่นการโจมตีที่ ต้องมีทั้งช่องโหว่ซอฟต์แวร์และการตั้งค่าผิดพร้อมกัน โดยไม่ต้องรองรับเต็มรูปแบบแบบ MulVAL
- **วัดสัดส่วนเส้นทางข้ามโดเมนด้วย k เส้นทางแทนเส้นทางเดียว** (k-shortest path) เพื่อตอบคำถาม “สัดส่วนของเส้นทางอันตราย” ได้ครบถ้วนกว่าการดูเฉพาะเส้นทางถูกสุด เพียงเส้นเดียวตามที่ทำในหัวข้อ 4.7
- **ตรวจสอบตัวสร้างกราฟจำลองกับ topology เครือข่ายจริง** เพื่อยืนยันว่าโครงสร้าง แบบโซ่ที่พบ (หัวข้อ 4.9) เป็นคุณสมบัติของเครือข่ายไฮบริดจริงด้วยหรือเป็นเพียงลักษณะ เฉพาะของตัวสร้างกราฟ

เอกสารอ้างอิง

[1] O. Sheyner et al., “Automated Generation and Analysis of Attack Graphs,” *IEEE S&P*, 2002.

[2] X. Ou et al., “MulVAL: A Logic-based Network Security Analyzer,” *USENIX Security*, 2005.

[3] S. Noel et al., “Efficient Minimum-Cost Network Hardening via Exploit Dependency Graphs,” *ACSAC*, 2003.

[4] T. Lengauer and R. Tarjan, “A Fast Algorithm for Finding Dominators in a Flowgraph,” *ACM TOPLAS*, 1979.

[5] J. Jacobs, S. Romanosky, B. Edwards, M. Roytman, and I. Adjerid, “Exploit Prediction Scoring System (EPSS),” presented at Black Hat USA, 2019. [Online]. Available: https://arxiv.org/abs/1908.04856

[6] FIRST, “Common Vulnerability Scoring System version 3.1: Specification Document.” [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[7] FIRST, “Exploit Prediction Scoring System (EPSS).” [Online]. Available: https://www.first.org/epss/

[8] MITRE, “Common Weakness Enumeration (CWE).” [Online]. Available: https://cwe.mitre.org/

[9] Center for Internet Security, “CIS Amazon Web Services Foundations Benchmark v3.0.0,” 2024.

ภาคผนวก ก

ตาราง Rubric ของ misconfiguration

ตารางนี้สกัดโดยตรงจากเอกสาร rubric ของ misconfiguration (18 แถว) เอกสารนี้คือที่ที่คน กำหนดเวกเตอร์ CVSS ให้แต่ละคลาสด้วยมือ พร้อมอ้างอิง CWE/CIS กำกับไว้ มีการตรวจสอบความถูกต้องสองชั้น ชั้นแรก `scripts/build_vectors.py --check` อ่านไฟล์นี้แล้วเทียบ กับ `data/fit/misconfig_vectors.json` ที่ใช้งานจริงว่าทั้งสองไฟล์ตรงกันทุกแถวหรือ ไม่ ป้องกันไม่ให้สอง source of truth แยกออกจากกันโดยไม่มีใครสังเกต ชั้นที่สอง `scripts/calibrate_rubric.py` นับว่าแต่ละ CVSS cell (ชุดค่า 8 metric ที่ซ้ำกัน) ใน ไฟล์ข้อมูล CVE จริงมี CVE กี่รายการรองรับ แล้วเขียนค่า `conf` กลับไปตามจำนวนนั้น

รายละเอียดเหตุผลรายเมตริกและอ้างอิง CWE/CIS เต็มรูปแบบของแต่ละแถวอยู่ในไฟล์ต้นฉบับ คอลัมน์ `conf` คือความมั่นใจของการกำหนดเวกเตอร์ ไม่ได้ควบคุมขนาด perturbation ในโค้ดแต่อย่างใด (ดูหัวข้อ 2.2.3) แถวที่มี `conf=low` (`ssrf-imdsv1-token`, `smb-signing-disabled`) คือ CVSS cell ที่มี CVE จริงรองรับน้อยกว่า 10 รายการตามที่ `scripts/calibrate_rubric.py` นับได้จริง (ตรวจสอบ 2026-10-13)

ตารางที่ 16 Rubric ของ misconfiguration ทั้ง 18 class

slug	เวกเตอร์ CVSS v3.1	p_source	conf
`s3-public-read`	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`	fitted	high
`s3-public-write`	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	fitted	med
`iam-trust-wildcard-principal`	`AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`	fitted	high
`iam-trust-missing-external-id`	`AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H`	fitted	med
`iam-passrole-excess-privilege`	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`	fitted	high
`aws-key-on-host`	`AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`	fitted	high
`ssrf-imdsv1-token`	`AV:A/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`	fitted	low
`sg-open-mgmt-port`	`AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N`	deterministic	high
`flat-vlan-no-segmentation`	`AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N`	deterministic	high
`shared-local-admin-password`	`AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H`	fitted	high
`smb-signing-disabled`	`AV:A/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`	fitted	low
`unauth-internal-service`	`AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	fitted	high

slug	เวกเตอร์ CVSS v3.1	p_source	conf
`cicd-oidc-broad-sub`	`AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`	fitted	med
`stale-service-account-cred`	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`	fitted	med
`vpn-dx-to-private-subnet`	`AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N`	deterministic	high
`federated-identity-trust`	`AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:N`	deterministic	high
`federated-group-overbroad-role`	`AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`	fitted	med
`weak-ssh-credentials`	`AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	fitted	high

ภาคผนวก ข

สคีมา edge list

ไฟล์กราฟ (ทั้งจำลองและจาก lab) ใช้สคีมาเดียวกัน ไม่ว่าจะเก็บเป็น JSON (node-link format ของ networkx) หรือ CSV แบบเรียบ (`attackpath/graph.py`) node คือคู่ `(resource, privilege)` เก็บเป็น tuple สองช่อง edge หนึ่งแถวมีคอลัมน์ดังนี้:

คอลัมน์	ชนิดค่า	ความหมาย
`src_resource`, `src_priv`	string	node ต้นทาง (resource, privilege)
`dst_resource`, `dst_priv`	string	node ปลายทาง
`kind`	`cve`, `misconfig`, `deterministic`	ชนิดของ edge
`ref`	string	รหัส CVE หรือ slug ของ misconfig class
`domain`	`onprem`, `cloud`, `cross`	โดเมนของ edge
`p`	float $\in (0, 1]$	ความน่าจะเป็นที่กำหนดโดย `weights.assign()`
`p_source`	`epss`, `fitted`, `deterministic`, `expert`	แหล่งที่มาของ `p`
`p_input`	string	ค่าที่ใช้คำนวณ `p` ตรง ๆ (เช่น `epss:CVE-...` หรือ `cvss:AV:N/...`)

node ที่เป็น entry หรือ target ถูกทำเครื่องหมายผ่านแอตทริบิวต์ `entry`/`target` บน node นั้นโดยตรง (ไม่ใช่คอลัมน์ของ edge) `graph.validate_structure()` ตรวจสอบ ว่าทุก node เป็น tuple สองช่อง, ทุก edge มี `kind`/`domain`/`ref` ครบ, กราฟแบบ hybrid มี edge ที่ `domain="cross"` อย่างน้อยหนึ่งเส้น และทุกคู่ (entry, target) ไปถึงกันได้ ส่วน `graph.validate_weights()` ตรวจว่าทุก edge มี `p_source` ที่

ถูกต้องและ p อยู่ในช่วง $(0, 1]$

ภาคผนวก ค
วิธีทำซ้ำผลการทดลอง

ซอร์สโค้ดทั้งหมดของโครงงานนี้อยู่ที่

`https://github.com/Vnntnn/Attack-path-management-and-choke-point-analysis-hybrid-infrastructure`

1. โคลน repository ข้างต้น แล้วติดตั้ง `uv` ตามด้วย `uv sync` ที่รากโปรเจกต์ (ใช้ Python 3.13 เท่านั้น)
2. ดึงข้อมูล EPSS + NVD: `uv run python scripts/pull_data.py` (แคชลงดิสก์ ถาวรใน `data/raw/`, รันซ้ำได้โดยไม่ดึงใหม่)
3. fit ฟังก์ชัน CVSS⟶EPSS: `uv run --group fit scripts/fit_weights.py` (ต้องมี `data/fit/cvss_epss_joined.csv` จากขั้นก่อน)
4. รันกริดการทดลองเต็ม (216 โครงสร้าง ✕ 4 โมเดลน้ำหนัก): `uv run python experiments/run.py --jobs 8` (ใช้เวลาประมาณ 30–35 นาที และจะปฏิเสธไม่รันถ้ามีไฟล์ `results/exp*.csv` เดิมอยู่แล้ว ต้องเติม `--force` เพื่อลบก่อนรันใหม่)
5. รัน exp 6 แยกต่างหาก: `uv run python -m experiments.exp6_fit_error`
6. สร้างตารางสรุปและกราฟ: `uv run --group report experiments/analyze.py` (เขียนตารางสรุปผล, `results/summary/*.csv`, `results/figures/*.png`)
7. สร้างรูปภาพประกอบรายงาน: `uv run --group report --group fit python scripts/make_report_figures.py` (อ่านจาก `results/summary/*.csv` และ `data/fit/residuals.csv` ที่สร้างไว้แล้วจากขั้นตอนก่อนหน้า เขียนตรงลง `docs/report/figures/` ต้องรันขั้นตอนนี้ใหม่ทุกครั้งที่ตัวเลขในขั้นตอนก่อนหน้า เปลี่ยน ไม่เช่นนั้นรูปภาพในรายงานจะไม่ตรงกับตัวเลขในตาราง)
8. ตรวจสอบตัวเลขในเอกสารกับผลจริง: `uv run python scripts/check_claims.py`

9. รันชุดทดสอบทั้งหมด: `uv run pytest -q`
10. ทำซ้ำ lab จริง: ดูขั้นตอนเต็มในเอกสารประกอบของ `lab/` (ต้องมี Docker; LocalStack และ Neo4j รีสตาร์ตแล้วต้อง `terraform apply` ใหม่เสมอเพราะ LocalStack community ไม่เก็บ state ข้ามการรีสตาร์ต)

กราฟจำลองทุกกราฟอ้างอิง seed และ `cve_pool_size`/`epss_snapshot` ที่บันทึกไว้ในแถวผลลัพธ์ทุกแถว การทำซ้ำกราฟชุดเดียวกันทุกบิตต้องใช้ snapshot ข้อมูลชุด เดียวกันกับที่ระบุไว้ (EPSS snapshot วันที่ 2026-08-25, NVD 33 หน้า) ไม่ใช่แค่ seed อย่างเดียว

ภาคผนวก ง

ความแตกต่างจากข้อเสนอโครงงาน (Deviation log)

รายการนี้สรุปทุกจุดที่ผลงานจริงต่างจากที่เสนอไว้ พร้อมเหตุผลของแต่ละจุด

- **GNS3 ⟶ เครือข่าย Docker ธรรมดา** `gns3` ที่มีในเครื่องเป็นเพียง GUI client ต้องมี display และ `gns3server` backend แยกต่างหาก ซึ่งรันแบบ headless ไม่ได้ในสภาพแวดล้อมนี้ `docker-compose` สองโฮสต์บนเครือข่าย flat เดียวกันให้ คุณสมบัติ reachability แบบเดียวกับที่ GNS3 จะจำลอง และตัวเครือข่าย flat เองก็*คือ* ตัว misconfig `flat-vlan-no-segmentation` ที่ต้องจำลองอยู่แล้ว
- **Nessus ⟶ nmap** ไม่มีรหัส activation ของ Nessus Essentials ลองใช้ OpenVAS เป็นทางเลือกแรกแต่ล้มเหลวเพราะพื้นที่ดิสก์ไม่พอสำหรับฐานข้อมูล ช่องโหว่ที่แถมมา (ไม่ใช่ปัญหา license) จึงเปลี่ยนเป็น `nmap` แทน ซึ่งให้ข้อมูลแค่ การเข้าถึงพอร์ตเท่านั้น ไม่มี CVE ประกอบเลย ผลคือกราฟ lab ไม่มี edge ชนิด `cve` แม้แต่เส้นเดียว ข้อจำกัดนี้เหมือนกันไม่ว่าจะเลือก OpenVAS แบบไม่มี feed หรือ nmap
- **AWS จริง ⟶ LocalStack** ทดลองจริงบนบัญชี AWS Academy Learner Lab แล้วพบปัญหาสองชั้น ชั้นแรกคือคำสั่งสร้าง role และ user ใหม่ใน IAM (`iam:CreateRole`/`iam:CreateUser`) ถูกบล็อกไม่มีเงื่อนไข เป็นข้อจำกัดของบัญชีประเภทนี้โดยเจตนา (IAM คือ Identity and Access Management ระบบจัดการสิทธิ์และตัวตนของ AWS ที่ role/user/policy ทุกตัวของบัญชีขึ้นอยู่กับมัน) ชั้นที่สองคือ `s3:GetBucketObjectLockConfiguration` ถูกบล็อกด้วย SCP ระดับองค์กร (Service Control Policy คือกฎที่ผู้ดูแลระดับองค์กรตั้งครอบทุกบัญชีลูกไว้ล่วงหน้า แก้ไขจากภายใน บัญชีลูกเองไม่ได้เลยแม้จะมีสิทธิ์ admin ในบัญชีนั้นก็ตาม) และแก้จาก Terraform ไม่ได้ เช่นกัน เพราะ misconfig ทุกคลาสในโครงงานนี้พึ่งการสร้าง IAM resource ล้วน การรันบน LocalStack จึงไม่ใช่ทางเลือกที่ถูกกว่าเฉย ๆ แต่เป็นทางเลือกเดียวที่ทำได้จริงบนบัญชี ประเภทนี้
- **จำนวนกราฟ: 200 กราฟตามข้อเสนอ ⟶ 216 โครงสร้าง (864 กราฟถ่วงน้ำหนัก)** ข้อเสนอระบุ “อย่างน้อย 200 กราฟ” กริดสุดท้ายมี 216 โครงสร้างที่แตกต่างกัน (27 เซลล์ปัจจัย × 8 ซ้ำ) ซึ่งเกินขั้นต่ำแม้นับเฉพาะโครงสร้างก่อนคูณด้วยโมเดลน้ำหนัก กริด ก่อนหน้า (135 โครงสร้าง) ถูกแทนที่เพราะนับโครงสร้างได้ไม่ถึง 200